IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 2 June 2025, accepted 12 June 2025, date of publication 18 June 2025, date of current version 26 June 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3580552*

RESEARCH ARTICLE

# State of Charge Estimation for Li-Ion Batteries: An Edge-Based Data-Driven Approach

**SESIDHAR DVSR[1], CHANDRASHEKHAR BADACHI[1], (Member, IEEE), CHANDRASHEKAR NAGAWARAM[2], PANDURANGA CHARY KONDOJU[3], C. DHANAMJAYULU[4], (Senior Member, IEEE), AND INNOCENT KAMWA[5], (Fellow, IEEE)**

[1]Department of EEE, Ramaiah Institute of Technology, VTU, Belagavi 590018, India
[2]ABB, Hyderabad, Telangana 500081, India
[3]RAE, Ford Motor Company, Dearborn, MI 48124, USA
[4]School of Electrical Engineering, Vellore Institute of Technology, Vellore 632014, India
[5]Department of Electrical Engineering and Computer Engineering, Laval University, Quebec, QC G1V 0A6, Canada

Corresponding authors: C. Dhanamjayulu (dhanamjayulu.c@vit.ac.in) and Innocent Kamwa (innocent.kamwa.1@ulaval.ca)

This work was supported in part by Canada National Sciences and Engineering Research Council through the Laval University under Grant ALLRP567550-21.

**ABSTRACT** The traditional machine learning approach requires substantial computational resources which are scarce in the embedded devices. Recently, the confluence of Edge computing with IoT has enabled resource constrained embedded devices to implement machine learning algorithms. TinyML, with its emphasis on integrating machine learning into embedded systems, seeks to move the end users away from high-performance machines, towards devices with limited resources and power. The present work investigates the estimation of state of charge (SoC) of Lithium Ion (Li-ion) batteries focussing on data-driven methodologies developed during the last five years. This paper mainly focusses on the relationship between dataset characteristics and data stationarity, exploring battery behaviour prediction and related dataset comprehension techniques. A Long Short-Term Memory (LSTM) network, a variant of Recursive Neural Networks (RNN), is utilised for SoC estimation. A 1C rating standard is implemented to comprehend the charge and discharge properties of a Li-ion battery. This study includes hardware evaluation as well as Monte Carlo simulation analysis in circuit component design. The experimental results provide a Mean Absolute Error (MAE) of 0.0630, a Mean Squared Error (MSE) of 0.0107, and a Root Mean Squared Error (RMSE) of 0.1033, confirming the efficacy of the proposed methodology. These results illustrate the accuracy and reliability of SoC estimation model. In conclusion, the proposed data-driven technique can lead to improved data security, reduced latency and cost in the estimation of SoC for Li-ion batteries.

**INDEX TERMS** Data-driven model, edge-computing, Li-ion battery, optimization, state-of-charge, training, Testing.

## NOMENCLATURE

| | |
|---|---|
| $\alpha$ | Learning rate. |
| $\beta_1, \beta_2$ | Exponential decay rates for moment estimates. |
| $\Phi_k$ | Input feature vector at time step $k$. |
| $\$$ | Loss function (Mean Squared Error). |
| $\sigma(\cdot)$ | Sigmoid activation function. |
| $\tanh(\cdot)$ | Hyperbolic tangent activation function. |
| $\tilde{m}_\epsilon$ | Bias-corrected first moment estimate. |
| $\tilde{r}_\epsilon$ | Bias-corrected second moment estimate. |
| $\widetilde{C}_k$ | Candidate cell state at time step $k$. |
| $b_c, b_f, b_i, b_O$ | Bias terms for respective gates. |
| $b_y$ | Bias term for output layer. |
| $C_k$ | Cell state at time step $k$. |
| $C_{k-1}$ | Cell state from previous time step. |
| $f_k$ | Forget gate activation at time step $k$. |
| $h_k$ | Hidden state at time step $k$. |
| $i_k$ | Input gate activation at time step $k$. |

The associate editor coordinating the review of this manuscript and approving it for publication was Xiaosong Hu.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

| | |
|---|---|
| $k$ | Numerical stability constant (e.g., $10^{-8}$). |
| $m_\epsilon$ | First moment estimate at iteration $\epsilon$. |
| $O_k$ | Output gate activation at time step $k$. |
| $r_\epsilon$ | Second moment estimate at iteration $\epsilon$. |
| $SoC_k$ | Estimated State of Charge at time step $k$. |
| $SoC_k^\star$ | True (target) State of Charge at time step $k$. |
| $V_{out}$ | Output weight matrix. |
| $W_\epsilon$ | Weights at iteration $\epsilon$. |
| $W_{c\Phi}$ | Weight matrices for candidate cell state. |
| $W_{f\Phi}$ | Weight matrices for forget gate. |
| $W_{i\Phi}$ | Weight matrices for input gate. |
| $W_{O\Phi}$ | Weight matrices for output gate. |
| $\Phi_k$ | Input vector at time step 'k'. |
| $\Phi_N$ | Input vector at time step 'k. |
| $\tau$ | Time constant. |
| $H_0$ | Null Hypothesis. |
| $H_1$ | Alternative Hypothesis. |
| $h_k$ | represents the hidden layer. |
| $I(k)$ | Battery Current. |
| $I_b$ | battery current. |
| $Q$ | charge level. |
| $SoC_t$ | SoC at time t. |
| $T(k)$ | Battery Temperature. |
| $V(k)$ | Battery Voltage. |

## I. INTRODUCTION

A committee on intergovernmental panel on climate change [1] underscores the urgent need for immediate action to peak greenhouse gas emissions before 2025 and decrease them by 43% by 2030 to limit global warming to 1.5 °C, a goal central to the Paris agreement, which unites nations in combating climate change and adapting to its consequences. To answer this issue, the fast development of electric vehicles (EVs) has garnered significant attention from academics and businesses throughout the last ten years. Lithium Ion (Li-ion) batteries are essential to these EVs' operation and are crucial to guaranteeing their dependability and safety. Li-ion batteries are a very competitive and promising choice because of their exceptional energy storage capacity, extended lifespan, efficiency, and environmentally benign operation [2]. Creating a Battery Management System (BMS) has become an essential bridge between the vehicle and its battery pack to efficiently monitor the condition of these batteries and prevent any potential damage. BMS play a pivotal role in addressing the challenge of accurately estimating the state of charge (SoC) of batteries, a crucial metric indicating the remaining power before recharge. Precise SoC assessment is essential for assuaging user anxieties regarding battery depletion and ensuring optimal battery performance. Achieving accurate SoC estimation is vital for extending the lifespan of vehicle batteries. However, it is very important to acknowledge that the precision of SoC estimation is subject to various influencing factors, which require thorough investigation.

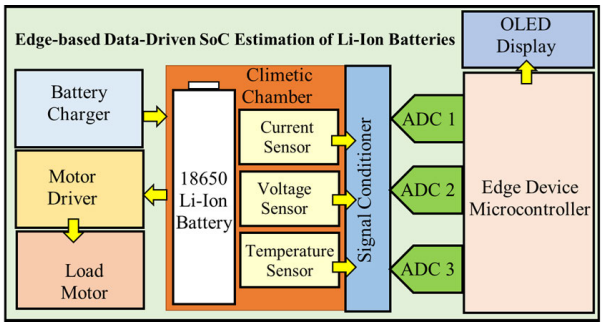


**FIGURE 1.** Block diagram of an Edge-based SoC estimation system.

### A. LITERATURE REVIEW

State of Charge (SoC) estimation for lithium-ion batteries (LIBs) has been approached through both model-based and data-driven methodologies. Each strategy offers trade-offs between physical interpretability, computational complexity, and real-time feasibility, especially for embedded battery management systems (BMS).

#### 1) MODEL-BASED APPROACHES

Model-based SoC estimation typically consists of four key stages: model selection, battery testing, parameter identification, and algorithmic implementation. The most prominent model categories include empirical models (EM), electrical equivalent circuit models (EECM), electrochemical models (ECM), and electrochemical impedance models (ECIM) [3].

*Empirical models (EM)* use curve-fitting or polynomial functions to approximate battery behavior. While easy to implement, their increased parameterization to improve accuracy results in computational overhead and reduced scalability for real-time applications.

*Electrical equivalent circuit models (EECMs)* approximate terminal voltage response using passive circuit components and are often derived from Hybrid Pulse Power Characterization (HPPC) datasets [4]. However, their reliance on simplified linear systems renders them less reliable under dynamic operating conditions.

*Electrochemical models (ECMs)* describe internal battery dynamics via partial differential equations grounded in physical laws. Though highly accurate, their complexity and computational demands restrict their use in real-time BMS [5].

*Electrochemical impedance models (ECIMs)* further extend ECMs by incorporating impedance spectroscopy but similarly suffer from high analytical burdens.

To support these models, battery testing protocols such as the Constant Current Discharge Test (CCDT) [6], Dynamic Stress Test (DST), and US06 urban driving cycle [7] are used to collect operational data. Parameter identification is then performed using algorithms like Genetic Algorithms (GA) [8] and Recursive Least Squares (RLS). Advanced filters such as the Cubature Kalman Filter (CKF) with the Improved Generalized Minimum Error Entropy (IGMEE) [9], and

Particle Filters (PF) [10], further improve estimation under noisy, nonlinear conditions.

#### 2) DATA-DRIVEN APPROACHES

In contrast, data-driven models aim to learn direct mappings from measurable signals—voltage, current, and temperature—to SoC, without requiring deep physical insight into battery chemistry. These models are inherently more flexible and have seen increasing adoption due to advancements in machine learning.

Various data-driven methods include:

- **Regression and classification methods:** Support Vector Regression (SVR), Random Forest (RF) [11], [12].
- **Probabilistic methods:** Gaussian Process Regression (GPR), Deep Belief Networks (DBN) [13], [14].
- **Hybrid methods:** Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Gravitational Search Algorithm (GSA) [15], [16], [17].
- **Fuzzy systems:** Fuzzy Logic (FL), Adaptive Neuro-Fuzzy Inference System (ANFIS) [18], [19].
- **Feedforward neural networks:** Backpropagation Neural Networks (BPNN), Extreme Learning Machines (ELM), Time Delay Neural Networks (TDNN), Deep Feedforward Neural Networks (DFFNN) [20], [21], [22], [23].
- **Recurrent neural networks:** Gated Recurrent Unit (GRU), Long Short-Term Memory (LSTM) [24], [25].

Among these, RNN-based architectures—particularly LSTM—have demonstrated strong temporal modeling capabilities, making them well-suited for sequential data like battery charge-discharge profiles.

### B. RESEARCH GAPS

Despite substantial progress in both model-based and data-driven SoC estimation techniques, several critical gaps remain:

1) **Model-based methods** are limited by their dependency on physical parameters and high computational cost, making them unsuitable for real-time, embedded environments.
2) **Data-driven approaches**, although accurate, are often trained and deployed on resource-rich platforms. Little attention has been given to deploying such models on ultra-low-power microcontrollers for real-time inference in edge environments.
3) **Online learning and adaptation** capabilities are rarely explored in embedded implementations. Most data-driven studies focus on offline model training without considering model drift or deployment under time-varying loads.
4) **Model interpretability and consistency of notation** across literature is inconsistent, which limits cross-comparison and practical implementation across platforms.

This work addresses these gaps by proposing a lightweight, LSTM-based SoC estimation model trained offline and deployed on a low-power edge platform (Arduino Nano 33 BLE Sense). The approach is validated on real battery datasets, quantized for embedded deployment, and benchmarked against accuracy and efficiency metrics, providing a practical and scalable SoC estimation solution.

### C. NOVELTY IN THIS WORK

While RNN-based SoC estimation using Long Short-Term Memory (LSTM) networks has been widely investigated in prior studies, their practical real-time deployment on edge microcontrollers remains a largely underexplored domain. This work bridges that gap by presenting a complete, operational pipeline—from LSTM model training and quantization to hardware-level deployment—on a memory-constrained microcontroller platform.

The novelty of this work does not lie in the LSTM architecture itself, but rather in its transformation into a deployable, edge-compatible SoC estimation system that operates independently of any cloud infrastructure. The following innovations distinguish this study:

- Deployment of a lightweight, quantized LSTM-based SoC model on a microcontroller using TensorFlow Lite, optimized for limited memory and computation.
- Empirical benchmarking of key performance indicators such as inference latency, memory footprint, and energy consumption on the embedded target device.
- Full elimination of cloud dependency, enabling low-latency, privacy-preserving SoC estimation for use cases like electric vehicles, drones, and off-grid renewable energy systems.

To the best of our knowledge, this is one of the first studies to demonstrate a fully functional, LSTM-based SoC estimator deployed on real-time edge hardware with quantitative performance evaluation. This effort advances the practical integration of intelligence into Battery Management Systems (BMS) by transitioning from theoretical or cloud-dependent frameworks to actual embedded implementations.

### D. CONTRIBUTIONS

The primary contributions of this study, in alignment with the research gaps identified in this Section as follows:

1) **End-to-end Edge Pipeline:** Design and implementation of a complete data-driven SoC estimation pipeline, including model training, quantization, and embedded deployment.
2) **Dataset Analysis and Feature Engineering:** Comprehensive preprocessing and stationarity analysis of the Li-ion battery dataset. A detailed correlation study is conducted to select optimal features for improved LSTM performance.
3) **Hardware Realization:** Development of signal conditioning circuits for current, voltage, and temperature

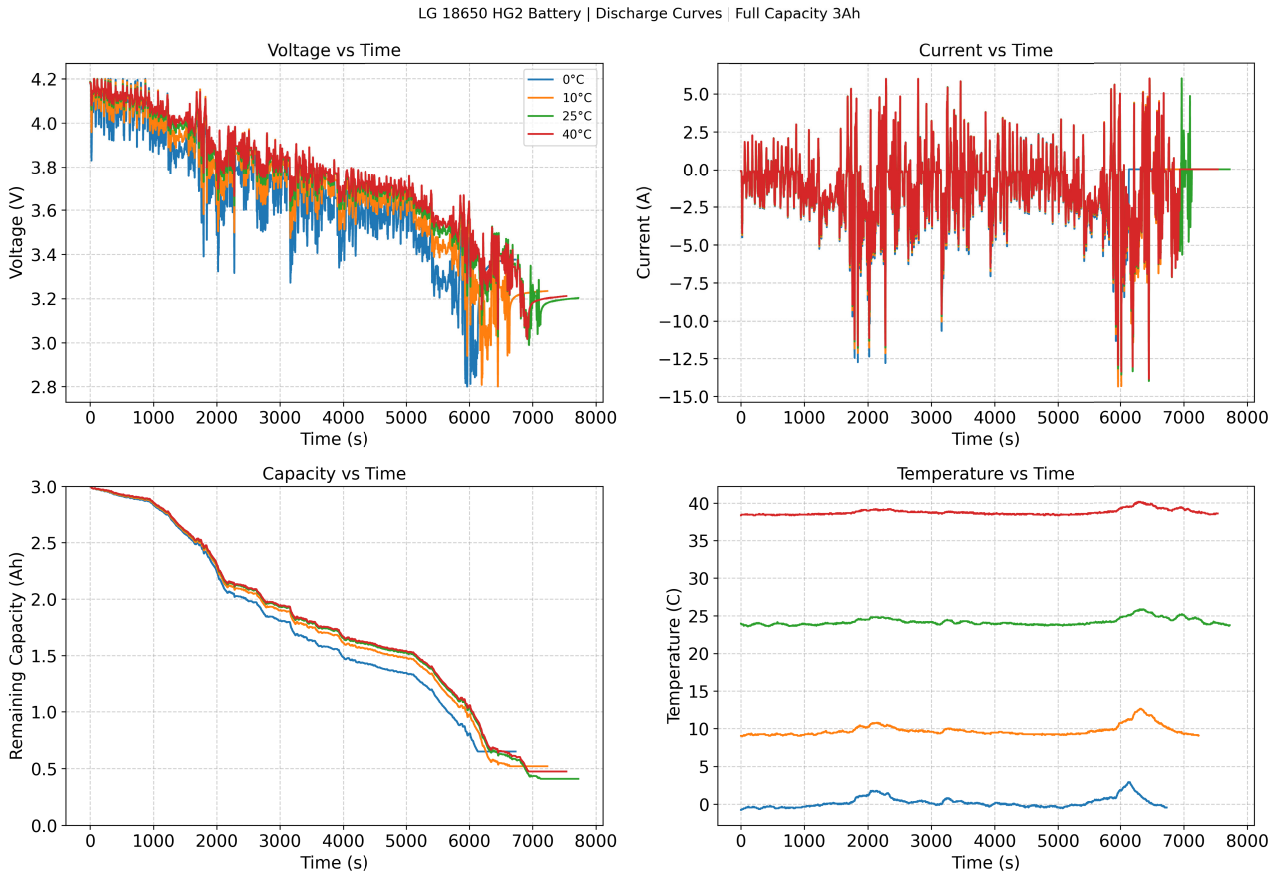


**FIGURE 2.** Battery Data from a Mixed1.csv applied on the 3Ah battery at the temperatures ranging from 0°$C$ (Blue), 10°$C$ (Orange), 25°$C$ (Green), and 40°$C$) (Red), Voltage, Current, Capacity and Cell Temperature.

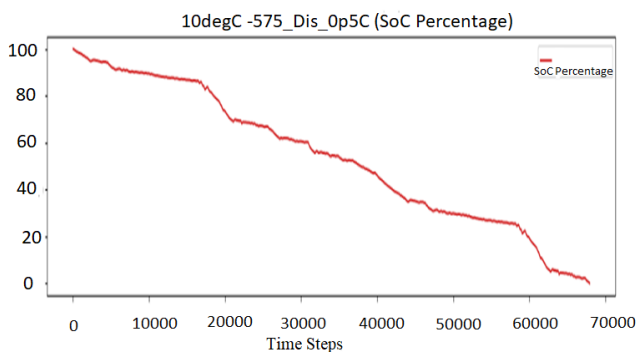


**FIGURE 3.** Discharge at 0.5C showing SoC Percentage vs Time at 10 °C Dataset.

acquisition. Monte Carlo analysis is used to evaluate the effect of sensor tolerance on model accuracy in a real-world setup.

The remainder of this paper is organized as follows:

Section II introduces the fundamentals of SoC estimation and explains the time-series preprocessing techniques. Section III describes the LSTM model architecture, training methodology using TensorFlow, and evaluation strategy. Section IV outlines the experimental hardware setup and real-time data acquisition procedures, performance metrics, embedded benchmarking results. Section V presents concluding insights.

## II. SoC: STATE OF CHARGE

SoC indicates the residual capacity of a battery, which is at a certain discharge rate.

### A. RESIDUAL CAPACITY VS ACTUAL CAPACITY

SoC is a ratio of its present battery residual capacity $Q_t$ and its overall available capacity $Q_o$. The SoC indicates to the user how much longer the battery can operate before it has to be charged or replaced. This can be expressed as:

$$SoC(t) = \frac{Q_t}{Q_0} \times 100\% \tag{1}$$

where:

SoC(t) represents SoC at time $t$. $Q_t$ represents the current charge level of the Li-ion battery at time $t$. $Q_0$ represents initial capacity of the Li-ion battery. The result is expressed as a percentage.

Battery SoC Equations: To determine the SoC at a given time $t$ SoC(t), we calculate it as the integral of the battery current, denoted as $I_b(\tau)$, from the initial time $t_0$ to $t$. This

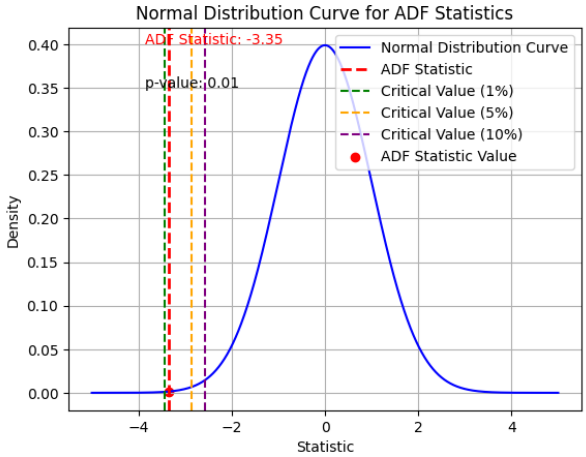


**FIGURE 4. Augmented Dickey-Fuller test results for stationarity analysis. The plot reports ADF statistic and p-value, used to validate the stationarity of time-series battery features before model training.**

represents the remaining power in the battery at time $t$.

$$Q_t = \int_{t_0}^{t} I_b(\tau)d\tau \tag{2}$$

The available capacity of the battery, represented as $Q_0$, is computed in a similar manner, integrating the battery current from $t_0$ to $t_2$.

$$Q_0 = \int_{t_0}^{t_2} I_b(\tau)d\tau \tag{3}$$

using these values, the SoC at time $t$ is calculated as a percentage, with $Q(t)$ as the remaining power and $Q_0$ as the overall capacity.

$$SoC(t) = \frac{\int_{t_0}^{t} I_b(\tau)d\tau}{\int_{t_0}^{t_2} I_b(\tau)d\tau} \times 100\% \tag{4}$$

It's important to note that a fully charged battery has an SoC of 100 %, while a completely discharged battery has an SoC of 0 % (the min allowable capacity limit). for instance, if a battery with the rated voltage of 12.6 V is fully charged, the SoC is 100 %. At 10.5 V, the SoC is 0 %. To maintain the battery at 40 % SoC, its voltage should be around 11.9 V.

"In this work, current is defined with respect to the battery terminals, where positive current denotes charging (current entering the battery), and negative current denotes discharging (current leaving the battery). This convention is consistently applied throughout all stages of the system, including data collection, feature engineering, and SoC model development."

### B. EDGE-BASED SoC ESTIMATION FRAMEWORK

SoC estimation primarily entails gathering crucial data from the battery pack, including Voltage, Current, Cell temperature, battery Charging/Discharging rate information, and Power. The corresponding time stamp of each operation is also captured. Using the captured parameters, the intelligent BMS can determine an optimum way for cell-balancing during an unbalanced charging/discharging pattern. It also helps in getting a control over thermal runaway to prevent cell overheating, and effectively handle any operational faults. BMS assists in estimating the SoC, state of health (SoH), state of power (SoP), and state of function (SoF). This sophisticated system could be utilized for transmitting the necessary data from the edge device onto the cloud whenever required, while updating the data into a local memory at the same time. Fig. 1 displays a comprehensive block-diagram of an edge-based data-driven SoC estimation system.

### C. BENCHMARK DATASET DESCRIPTION

A typical battery data set used for the initial study is taken from Mendeley Data repository [26], LG 18650HG2 Li-ion Battery data. the data is generated on a new 3 Ah LG HG2-18650 battery placed inside an 8 cu.ft thermal chamber, set to initial testing conditions with 75 A and 5 V testing jig manufactured by Digatron Universal Battery Testing with an 0.1% accuracy in voltage and current. This data set trains the networks, which is shown in Fig. 2. which can be represented as.

$$Dataset = (\Phi_1 : SoC_1^{\star}); (\Phi_2 : SoC_2^{\star}); ..; (\Phi_N : SoC_N^{\star}) \tag{5}$$

here, $\Phi_k$ = Input vector at time step 'k', $SoC_k^{\star}$ = Truth value of 'SoC' at time step 'k'. The input vector is defined as

$\Phi_k = [V(k); I(k); T(k)]$

where $V(k), I(k)\&T(k)$ represent the battery's Voltage, Current, and Temperature respectively at time step k.

### D. STEPS IN SoC ESTIMATION

Before attempting to estimate the SoC from a time series dataset of 18650 Li-ion batteries using machine learning or deep learning models, it is crucial to perform a number of initial evaluations to confirm the dataset's quality and applicability. Thoroughly examining and preparing the dataset is essential for accurately estimating the SoC.

Data cleaning is the first step in the process, when unnecessary or duplicate data items that might possibly bias the analysis are removed. Addressing missing values is another essential aspect, where methods such as interpolation, deletion, or imputation are employed based on the circumstances. Moreover, it is essential to carefully examine the data for any anomalies or outliers and choose the best appropriate course of action for each, such as adjusting the data, excluding it, or doing additional study.

Data exploration entails decomposing the original time series into its constituent parts, including the trend, seasonality, and residual components. This analysis aids in understanding the basic patterns and changes in the battery's SoC capacity over time, which are crucial for accurate prediction and subsequent analysis. The Fig. 3, graphically shows the data, on y-axis it is SoC capacity and the x-axis indicating time (as indicated by the 'Time Stamp' column). These graphs provide immediate insights into the observations in SoC capacity over time, for a 10degC 0.5 C discharge cycle.

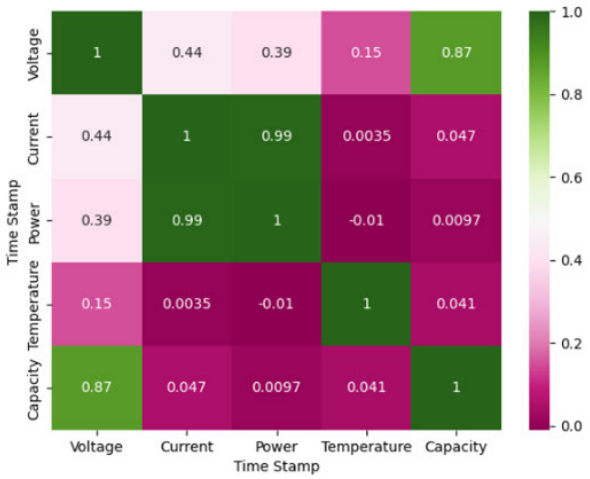


**FIGURE 5.** Feature correlation matrix for the battery dataset.

During the Feature Engineering phase, the time series data is examined to identify significant attributes such as voltage, current, temperature, and cycle count. These attributes are vital markers of the SoC. Furthermore, by using domain expertise, it is feasible to develop novel functions that enhance the precision of the models. These features may involve evaluating the patterns of charging and discharging, taking into account the time of day, and including statistical characteristics such as moving averages or windowed variances.

Model Validation involves dividing the dataset into separate training and testing sets, or use cross-validation procedures, in order to get impartial and efficient evaluations of the model. Ensuring that the testing set includes data from several time periods or settings is crucial to evaluate the model's ability to perform consistently and accurately in different contexts. Ultimately, in the Model Selection step, appropriate machine learning or deep learning models are chosen based on the specific job needs and data characteristics. Common choices for time series forecasting include LSTM networks for deep learning and regression trees for machine learning. By utilising ensemble methods, it is possible to enhance the accuracy of predictions by amalgamating many models to provide improved outcomes in comparison to utilising a solitary model in isolation.

### 1) TIME SERIES ANALYSIS

Examine the time series data for stationarity (e.g., by applying the ADF test) and, if necessary, apply differencing. Divide the time series into components related to trend, seasonality, and residuals, if any. ADF test: The Augmented Dickey-Fuller (ADF) test is a statistical test used to determine whether a time series dataset is stationary or non-stationary. If the time series has a unit root and it is non-stationary, it is considered as Null Hypothesis ($H_0$). If the time series does not have a unit root then it is called stationary or Alternative Hypothesis ($H_1$). ADF Statistic: Fig. 4 is shown as a vertical red dashed line on the plot. It represents the position of the test statistic on the normal distribution curve at $-3.3549$. If the test statistic is less than the critical value, we reject the null hypothesis and accept the Alternative Hypothesis, indicating the series is stationary. If a p-value less than 0.05 that indicates strong evidence against the Null Hypothesis, suggesting that the data is stationary. In our case, the p-value is 0.01263. This implies that the data does not exhibit a significant trend over time, which is crucial for many time series analyses and forecasting tasks.

### 2) MITIGATION OF NON-STATIONARITY EFFECTS

The ADF test assessed the dataset's stationarity. The ADF statistic and low p-value indicate that the series under consideration meets the stationarity assumption during the observed sample period. Data stationarity is essential for accurate prediction models, especially in time series forecasting. Degradation mechanisms include capacity fading, raise in internal resistance and thermal instability change LIB' fundamental characteristics over time in real-world applications. If uncontrolled, these degradation processes generate system non-stationarities, which can lower model predictability and resilience. The framework should address these difficulties by including long-term non-stationarity provisions. Integrating age-aware augmentation, adaptive model re-training, and frequently re-evaluating stationarity scenarios may increase the model's battery aging resilience. Experimental datasets that track battery aging or synthetic degradation trajectories can also verify resilience. By addressing non-stationary behaviours and integrating mechanisms to maintain model adaptability over the battery's operational lifetime, the suggested methodology improves its practicality and resilience in real-world battery health monitoring and prognostic applications.

### 3) CORRELATION ANALYSIS

In order to mitigate issues associated to multi-collinearity, it is necessary to focus on characteristics that exhibit a high degree of correlation. Correlation analysis identifies the relationships between the target variable, Capacity, and other independent variables. Create a list called 'chosen columns' that consists of the names of the columns to be included in the correlation analysis. Retrieve these names from the first row of the dataset. Use the data frame's technique to calculate the correlation matrix. Visualize the correlation matrix as a heatmap using Seaborn's heatmap function, with the correlation coefficients shown as annotations. This process will provide a heatmap visualization of the correlation matrix, offering insights into the pairwise correlations among the chosen columns in your dataset. Colors such as dark green imply positive correlations, but more intense colors like pink represent negative correlations. The saturation of the color corresponds to the magnitude of the association. Fig. 5 displays the correlation heat map for the utilized battery dataset.

RNN based deep learning methods, such as Long Short-Term Memory (LSTM) networks, are highly beneficial

for analysing non-stationary time-series data. RNNs are a suitable choice for modelling time series data, especially in cases where the data is non-stationary. This is because RNNs excel in recognising temporal correlations and patterns in sequential data.The user's text is simply a backslash character. Hence, it would be advantageous to investigate RNN-based deep learning techniques, such as LSTM networks, for the purpose of estimating the SoC using the time series dataset of your 18650 Li-ion battery. Although the data is not stationary, these models are able to effectively predict the data and capture its dynamic nature.

## III. ARCHITECTURE OF THE RNN-LSTM MODEL

RNNs are artificial neural networks that apply a wide range of time series data analysis tasks. Classical RNNs often encounter problems with long-term dependencies, such as the gradient either exploding or vanishing during back-propagation. Due to their high success rate, RNNs - LSTM excel at various applications. The RNN LSTM exceptional in capturing the relationship between input and output sequences, effectively representing intricate nonlinear dynamic systems.

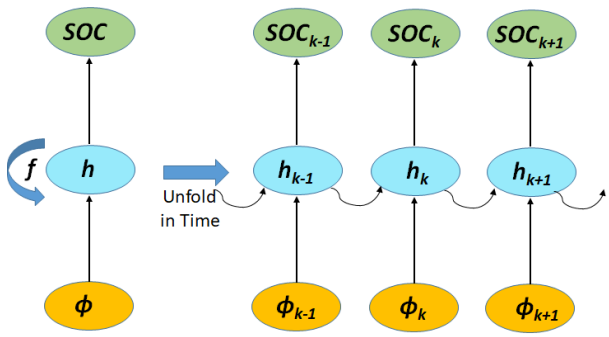


**FIGURE 6.** On the left side is the architecture of RNN and on the right is the architecture of RNN that is unfolded in time. The input vectors are given by $\phi_k = [V_k, I_k, T_k]$, where $V_k, I_k, T_k$ denotes voltage, current, and temperature of Li-ion battery at $k^{th}$ time step. $h_{k-1}$ represents the hidden layer of LSTM RNN at $(k-1)^{th}$ time step.

### A. RNN-LSTM BASED SoC ESTIMATION METHODS

Several studies have proposed Recurrent Neural Network (RNN) based models—particularly Long Short-Term Memory (LSTM)—to address the nonlinearities and temporal dependencies in Li-ion battery SoC estimation.

The work in [27] presents a real-time method for SoC estimation using advanced architectures like Deep Neural Networks (DNN), Artificial Neural Networks (ANN), and GRU-RNN. Both experimental and MATLAB/Simulink simulations were employed, where DNN models achieved mean errors below 0.5% and maximum errors under 2.4%.

In [28], a novel Parallel Artificial Neural Network (PANN) with BiLSTM layers improves SoC estimation accuracy by 1.5 to 3 times over conventional RNNs on public datasets. The model is also evaluated under extreme conditions to validate its robustness in real-world battery packs, with a reported MSE of 0.0298%.

Paper [29] proposes an extended-input LSTM (ELSTM) combined with adaptive multi-timescale identification and adaptive singular value decomposition-transformed unscented Kalman filtering (ASVD-UKF). The model achieves extremely low error values—MAE: 0.0939%, RMSE: 0.1074%, and MAPE: 0.1257%.

The authors in [30] employ an LSTM-RNN with an Adaptive Extended Kalman Filter (AEKF) for SoC estimation using 30-second time-series data. The model outperforms Coulomb counting and open-loop DNN methods with a maximum error of 2.5%, RMSE below 1.01%, and MAE under 0.64%.

In [31], the EI-LSTM-CO model integrates extended input and constrained output layers. The input is enhanced by a sliding window voltage average, while the output is stabilized using an AhI-based state flow mechanism. Validated on $LiFePO_4$ datasets, the model achieves RMSE and MAXE below 1.3% and 3.2%.

A robust IndRNN-based model is introduced in [32], trained independently on diverse Lithium Nickel Cobalt Aluminium Oxide (NCA) datasets across temperature variations. The model outperforms GRU and LSTM with RMSE: 0.7633%, MAE: 0.6389%, and max error of 2.2398%.

The hybrid approach in [33] combines Unscented Kalman Filters (UKF) with LSTM networks to harness both model-driven dynamics and data-driven accuracy. The RMSE remains within 1–2.3% across temperature conditions, showcasing strong generalization under varying operating states.

In [34], an AdaBoost.Rt-LSTM ensemble learning algorithm is developed to estimate both SoC and SoH for retired batteries. The algorithm adapts to spatio-temporal features across the battery lifecycle, yielding average absolute errors of 1.54% (SoC) and 1.48% (SoH), and RMSE of 2.15% and 2.02%, respectively.

The comparative study in [35] evaluates four RNN variants based on accuracy, aging adaptability, and robustness to measurement uncertainty. The Analytic Hierarchy Process (AHP) is used to guide BMS designers in selecting the optimal online SoC estimation model.

#### 1) DISCUSSION ON OFFLINE-ONLINE BLENDING STRATEGIES

While the above approaches demonstrate strong predictive performance, most rely on fully offline-trained models deployed without post-deployment adaptation. Recent advancements in blended learning paradigms, which combine offline pretraining with online fine-tuning, offer promising avenues for dynamic SoC estimation.

For instance, hybrid frameworks that initialize models via offline training and adapt weights online through continual learning or meta-learning strategies [41], [42] have shown improved robustness to distributional shifts and battery aging. These methods support gradual adaptation to evolving battery characteristics without the need for full retraining.

However, such online-update mechanisms typically require additional memory and processing overhead, which

**TABLE 1.** Comparison of RNN-Based SoC estimation models.

| Reference | MAE (%) | RMSE (%) | MAXE (%) |
|---|---|---|---|
| [27] | <0.5 | — | <2.4 |
| [28] | — | 0.0298 | — |
| [29] | 0.0939 | 0.1074 | — |
| [30] | 0.64 | 1.01 | 2.5 |
| [31] | — | <1.3 | <3.2 |
| [32] | 0.6389 | 0.7633 | 2.2398 |
| [33] | — | 1–2.3 | — |
| [34] | 1.54 | 2.15 | — |

may be unsuitable for ultra-constrained edge environments. In contrast, our approach adopts a fully offline-trained, quantized LSTM model optimized for real-time inference on microcontrollers with limited resources. While this limits post-deployment adaptability, it ensures energy efficiency, privacy, and low latency, aligning with the design constraints of edge-based battery management systems.

### B. OVERVIEW OF LONG SHORT-TERM MEMORY (LSTM)

The LSTM unit mainly consists of four components, they are the Memory Cell, the Input gate, the Output gate and the Forget gate. An LSTM unit shown in the Fig. 6, is the fundamental building block of LSTM networks.

#### 1) THE MEMORY CELL

The Memory cell is a key part of the LSTM unit responsible for storing and maintaining information over time. The core functionality of the LSTM unit is its ability to remember information over extended time intervals. This is achieved through the cell, which stores information and ensures it remains available for future use.

$$\begin{aligned} C_k &= (f_k * C_{k-1} + i_k * \widetilde{C_k}) \\ \widetilde{C_k} &= tanh(W_{c\Phi}\Phi_k + W_{hc}h_{k-1} + b_c) \end{aligned} \tag{6}$$

#### 2) THE FORGET GATE

The forget gate plays a critical role in deciding what information to keep and what to discard from the previous state. For this, a '$\sigma$' (sigmoid) function is used which states the importance of the cell. It does this by comparing the previous state with the current input and assigning a value between 0 and 1. A value close to 1 implies that the information is important and should be retained, while a value close to 0 signifies that the information should be discarded. The output '$f_k$' is determined by combining the current input '$\Phi$', with the hidden state '$h$' of the previous timestamp, and the bias '$b$'. '$W_f$' is the weights of respective states.

$$f_k = \sigma(W_{f\Phi}\Phi_k + W_{fh}h_{k-1} + b_f) \tag{7}$$

#### 3) THE INPUT GATE

This gate controls the flow of new information into the cell. Similar to the forget gate, the input gate determines which pieces of new information should be incorporated into the current state. It uses a similar mechanism as the forget gate to make this decision.

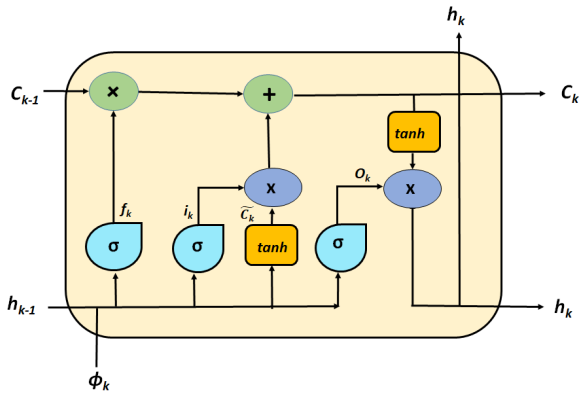


**FIGURE 7.** High-level structure of the LSTM model used for SoC prediction.

$$i_k = \sigma(W_{i\Phi}\Phi_k + W_{i,h}h_{k-1} + b_i) \tag{8}$$

#### 4) THE OUTPUT GATE

The output gate regulates which information from the current state is passed to the network's output. The output gate governs which parts of the information in the current state should be sent to the network's output. It assigns values between 0 and 1 to the information, taking into account both the previous and current states.

$$O_k = \sigma(W_{O\Phi}\Phi_k + W_{O,h}h_{k-1} + b_O) \tag{9}$$

It is evident that the same function represents each of the three gates. All that differs are the weights and biases. The input gate and forget gate are used to update the state of the cell. After that, a *tanh* function that restricts the cell state between $-1$ and 1 and the output gate determine the hidden state of the current time step.

$$h_k = O_k \times tanh(C_k) \tag{10}$$

The automated computation of back propagation in this framework allows for the quick development and testing of different network architectures. TensorFlow's basic LSTM cell, was incorporated into the constructed networks. The following composite function may be used to describe it: The initial hidden state, denoted as $h_0$, is initialised as a matrix of zeros. The symbol '$\sigma$ represents the sigmoid function. The variables $i, f, o$ and $c$ represent the input, forget, output gates, and memory cell, respectively. They are considered as gates because they are kind of sigmoid functions that may be zero-valued and may impede the flow of data to the subsequent processing node. The network weights (denoted by $W$) assoviated with each gate are distinct. Subscripts of $W$, which represents the transformation between the two corresponding components, which are $W_o$ for the input/output gate matrix and $W_{hi}$ for the hidden-input gate matrix. A bias, $b$, is added to the matrix multiplication at each gate to enhance the network's capacity to match the data. The last fully-connected layer applies a linear transformation on $h_k$,

the hidden state tensor, in order to provide the $k^{th}$ estimated SoC value, calculated as:

$$SoC_k = V_{out} h_k + b_y \tag{11}$$

where $V_{out}$ and $b_y$ denotes fully-connected layer's weight matrix and bias, respectively.The difference between the estimated and measured SoCs of the LSTM-RNN is best represented by the loss function: $ produced at the conclusion of each forward pass.

$$\$ = \sum_{i=1}^{N} \frac{1}{2}(SoC_k - SoC_k^{\star})^2 \tag{12}$$

where $N$ is the length of the sequence and $SoC_k$ as well as $SoC_k^{\star}$ are the estimated and ground truth values of SoC at time step $k$, respectively. A forward pass starts when the network is given training data and ends when errors and total loss are computed, as well as SoC estimations at each time step. During a training epoch, which comprises one forward pass and one backward pass, the network weights $W$ and biases $b$ are modified. In order to do this, we updated the network weights and biases depending on the gradient of the loss function using an optimization technique called Adam.

$$\begin{aligned} m_\epsilon &= \beta_1 m_{\epsilon-1} \nabla \$(W_{\epsilon-1}) \\ r_\epsilon &= \beta_2 r_{\epsilon-1} \nabla \$(W_{\epsilon-1})^2 \\ \tilde{m_\epsilon} &= m_\epsilon/(1-\beta_1^\epsilon) \\ \tilde{r_\epsilon} &= r_\epsilon/(1-\beta_2^\epsilon) \\ W_\epsilon &= W_{\epsilon-1} - \alpha \frac{\tilde{m_\epsilon}}{\tilde{r_\epsilon} - k} \end{aligned} \tag{13}$$

where:

$\alpha = 10^{-4}$ which is Training Step Size

Decay rates are set as $\beta_1 = 0.9$ and $\beta_2 = 0.999$

$\$ = Lossfunction$

$k$ is a constant term set to $10^{-8}$.

$W_\epsilon$ is the matrix of network parameters during the current training epoch and may be used as a placeholder for $W_{\Phi i}$, $W_{hi}$, $W_{\Phi f}$ and so on. These gate matrices, as well as the output weight matrix, $V_{out}$, and biases, are all initialised with a normally distributed random number generator with a mean of 0 and a standard deviation of 0.05. Only during training, to do a forward and backward pass to continually update the network weights until a convergence criterion is fulfilled. The network self-learns its weights with the backward pass, providing considerable gains over previous SoC estimate algorithms that involve the time-consuming creation and pasteurization of hand-engineered battery models.

During training, a forward and backward pass is used for regularly updating the network weights until a convergence condition is met. With the aid of the backward pass, the network self-learns its weights, providing a significant advance over earlier methods for predicting SoC, which required the challenging design and parameterized of hand-engineered battery models.Because the network parameters were previously learnt during training, just a forward pass during validation is required to provide the predicted SoC values at each time step. A reverse pass is not required.

Because the LSTM-RNN's forward pass consists of many matrix multiplications, it has a lower computational overhead once trained. In comparison, this uses less processing resources than other algorithms that can do identical functions. The LSTM-RNN can encode a battery's characteristic behaviour at a variety of ambient temperatures, eliminating the need for memory to hold numerous parameters for variable ambient temperatures, which is a task that older battery models often undertake. As a result, LSTM-RNN is a great alternative for estimating over a large number of cells in a battery pack.

The training dataset is composed of numerous concatenated distinct driving cycles; when acquired, it typically contains more than 100,000 time steps. A sequence of this length cannot be stored in our GPU memory during training. As a result, the approach employed to train LSTM-RNNs includes feeding the models one batch of the sequence at a time. When the batch length, $\widetilde{N}$, is less than the entire training sequence length, $N$ i.e., ($\widetilde{N} < N$), the LSTM cell is said to be unrolled in time for $\widetilde{N}$ steps. When the time constant of the intrinsic dynamics inside the sequence is less than $\widetilde{N}$, the LSTM-RNN can still capture the time dependencies. Several error functions are used to evaluate the estimated performance of our networks. The Maximum Error (MAXERR), standard deviation of errors (Std Dev), Root Mean Square Error (RMSE), mean square error (MSE) and mean absolute error (MAE) are among them. The memory cell $C_k$, which houses the long-term dependencies, is a feature of the LSTM cell, whose schematic depiction is seen in Fig. 7. The Input, Output, and Forget gates enable the LSTM to write new information to the memory cell or to forget old information during training. Fig. 8 gives the visualization of how the neural network looks.

### C. MODEL ARCHITECTURE, TRAINING PIPELINE, AND HYPERPARAMETER SELECTION

The model employed for SoC estimation is based on a Sequential LSTM architecture, implemented using TensorFlow and Keras. The input to the model is a sequence of time-series measurements, including voltage, current, temperature, and power, normalized to ensure feature scale consistency. The architecture comprises the following layers:

- An LSTM layer with 16 hidden units and "return_sequences=True", enabling the capture of temporal dependencies across short time windows of sequential input.
- Two Dense layers with 16 and 8 units respectively, both using ReLU activation functions to model non-linear relationships in the data.
- A final Dense output layer with one neuron and a linear activation function to produce continuous SoC predictions.

The model is compiled with the Huber loss function, which combines the robustness of MAE with the smooth gradient

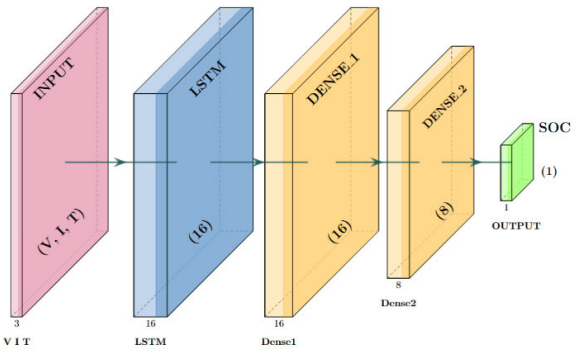


**FIGURE 8.** Neural network structure of the LSTM Model.

characteristics of MSE, making it suitable for regression tasks involving occasional outliers. Optimization is handled using the Adam optimizer with a fixed learning rate of 0.001. In addition to the loss, the model tracks MSE, MAE, and RMSE as evaluation metrics during training.

### D. VALIDATION STRATEGY AND EARLY STOPPING CRITERIA

The validation strategy employed in this study is based on a hold-out split approach rather than K-Fold cross-validation. Specifically, the test dataset is partitioned into an 80:20 ratio using "train_test_split", where 20% of the test data is reserved for validation during model training. This ensures that the models generalization capability is evaluated on unseen data while avoiding the computational overhead associated with full cross-validation. The hold-out method is suitable for the available dataset size and yields consistent results across multiple runs due to the use of a fixed random seed. To further enhance training robustness, two callbacks were integrated into the training pipeline:

- EarlyStopping: Monitors the validation loss and halts training if no improvement is observed for 20 consecutive epochs. This prevents overfitting and reduces unnecessary training cycles. The parameter "restore_best_weights=True" ensures that the final model retains the weights from the epoch with the lowest validation loss.
- ReduceLROnPlateau: Dynamically adjusts the learning rate during training. When the validation loss fails to improve for 5 epochs, the learning rate is reduced by a factor of 0.5. This allows finer convergence and prevents the optimizer from overshooting during later stages of training. To mitigate overfitting, learning rate decay via "ReduceLROnPlateau" was employed, which adaptively reduces the learning rate during stagnant validation performance, thereby improving convergence stability.

Together, these strategies contribute to a stable and efficient learning process, ensuring that the model achieves optimal generalization while minimizing overfitting. Table 2 shows the hyper parameters and training configuration of the model.

**TABLE 2.** LSTM model training configuration and hyperparameters.

| Parameter | Value / Description |
|---|---|
| Model Type | Sequential LSTM |
| LSTM Units | 16 |
| Dense Layers | Two (16 units and 8 units) |
| Activation Functions | ReLU (hidden), Linear (output) |
| Loss Function | Huber Loss |
| Optimizer | Adam |
| Learning Rate | 0.001 |
| Batch Size | 256 |
| Epochs | 100 (EarlyStopping applied) |
| Validation Split | 20% from test set |
| Sequence Length (Timesteps) | 1 |
| Evaluation Metrics | MSE, MAE, RMSE |
| Early Stopping | Patience = 20 |
| Restore best weights | True |
| Learning Rate Scheduler | ReduceLROnPlateau |
| Total Training Time | ~0.32 hours (recorded via timer) |

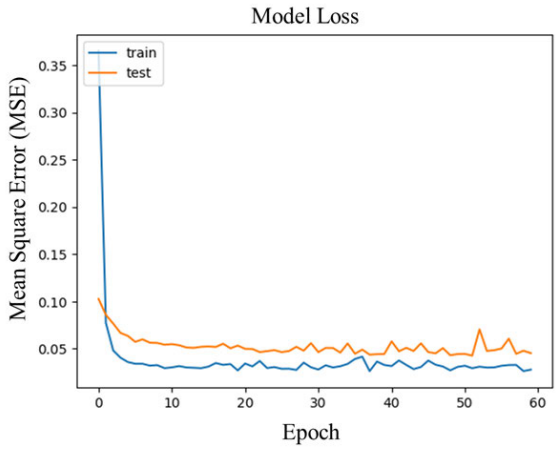


**FIGURE 9.** Training and validation loss (Mean Squared Error) curves for the LSTM model plotted over 100 epochs. The decreasing validation curve indicates generalization of the model on unseen data.

Fig. 9 displays the MSE: training and validation (test) loss curves for the LSTM model over the epochs. The significant decline in loss throughout the early epochs indicates that the model is successfully acquiring understanding based on the data. The selected number of epochs seems suitable, considering the early stabilization of loss values. Increasing the number of epochs in the training process yields no extra advantages since the loss remains steady. The fact that the training and validation loss curves are very close to each other suggests that the model can easily adapt to new data. This indicates that the model can perform well on fresh data and is not overfitting to the training set.

The chart in Fig. 10 displays the MAE curves for the LSTM model throughout the training and validation (test) phases over the epochs. The blue line, representing the MAE during training, starts at a high value and then rapidly decreases during the initial epochs. The validation MAE, shown by the orange line, exhibits a sharp decrease in the early epochs, suggesting that the model swiftly learns to minimize errors during training and validation. The consistent and enduringly low MAE values indicate that the model sustains its performance without any notable decrease during the training phase. The proximity between training and validation MAE

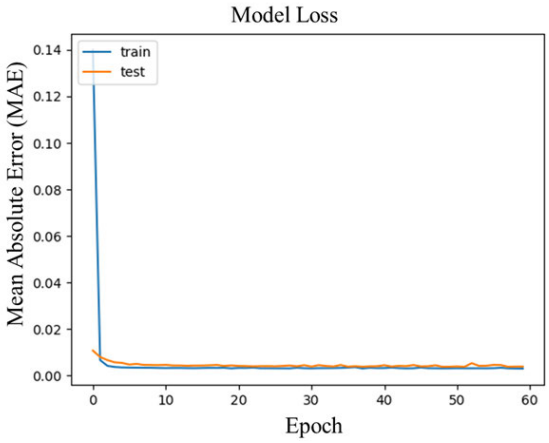


**FIGURE 10.** Mean Absolute Error (MAE) curves for both training and validation sets.

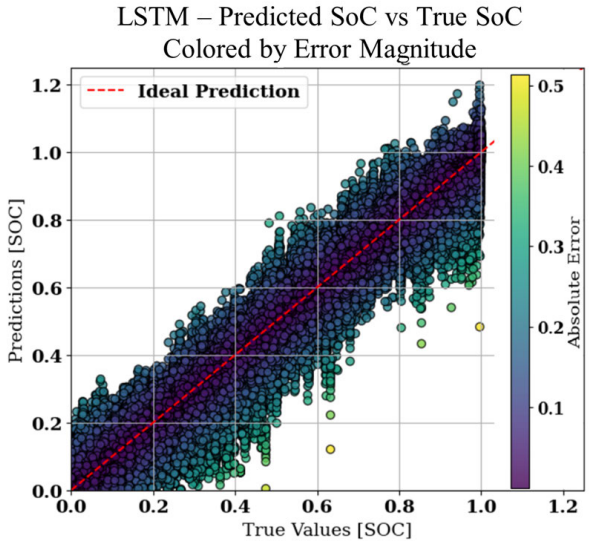


**FIGURE 11.** Scatter plot showing the performance of an LSTM (Long Short-Term Memory) model in predicting the SoC of a battery.

curves suggests strong generalization capability, indicating the model is not overfitting. These findings indicate that the model is not suffering from overfitting and can function effectively on unfamiliar data—the consistently low MAE values observed across the dataset.

The plot in Fig. 11 presents a scatter distribution of actual versus predicted SoC values, overlaid with a diagonal reference line representing ideal predictions. Most data points cluster tightly around this diagonal, indicating high model accuracy in estimating SoC. Notably, the plot employs a color map to visually encode the prediction error, where the color intensity of each data point corresponds to its deviation from the ideal prediction line. Samples that lie closer to the diagonal are rendered in cooler tones (e.g., blue), while those with higher prediction errors appear in warmer tones (e.g., yellow), effectively highlighting regions of reduced precision. This visual gradient aids in interpreting localized prediction performance across the dataset. Quantitatively, the model demonstrates strong predictive capability, as reflected by key error metrics: a MAE of 0.0630, indicating the average magnitude of prediction error; a MSE of 0.0107, capturing the average squared deviation; and a RMSE of 0.1033, representing the standard deviation of prediction errors. Together, the color-mapped scatter plot and statistical metrics affirm the model's reliability in forecasting SoC across a wide operating range.

The topmost plot in Fig.12 compares the projected SoC estimated by the LSTM model with the actual SoC over several time intervals. The bottom figure illustrates the discrepancy (the difference between the projected and actual SoC) about increasing time steps. The LSTM model has outstanding accuracy in forecasting SoC for a wide range of time intervals, as seen by the tight correspondence between the projected and real SoC curves. Most of the errors are concentrated around zero, suggesting a high accuracy level in the predictions. Several notable increases in inaccuracy correspond with the transitions occurring at specific time steps, as noted in the plot. The variations in inaccuracy suggest that the model has difficulties at particular transitions or sudden changes in SoC. When we merge different datasets, we can observe sudden and significant fluctuations in the data. The model may not encounter these sudden fluctuations in the data during real-time execution. Consequently, these increases can be disregarded. The error remains consistently low and constant throughout most of the steps, except for occasional power surges.

An advanced error analysis was conducted to rigorously assess the LSTM-based SoC estimation model, utilizing the prediction error distribution from the test dataset. Figure 13 illustrates that the majority of prediction errors are close to zero, signifying negligible bias in the model's estimations. The distribution has some left-skewness, rendering it nearly symmetric; however, it may underrepresent the actual SoC, particularly at low temperatures or low SoC levels. The histogram indicates that the majority of errors fall within the range of $-5\%$ to $+5\%$ SoC, satisfying the practical accuracy standards for embedded BMSs. In infrequent instances, outlier errors of $-40\%$ and $+20\%$ were documented, likely attributable to significant battery degradation or harsh climatic conditions. The minimal spread and concentration of errors near zero indicate that the LSTM model is resilient and generalizable across diverse operational conditions.

The suggested LSTM model optimally balances prediction accuracy and deployment practicality in resource-constrained situations like the Edge -Microcontrollers. Table 2 shows that the suggested model has a lower MAE (0.0630%) and RMSE (0.1033%) than GRU-RNN, BiLSTM, and E-LSTM versions. Complex ensemble models and deep recurrent architectures are inappropriate for real-time embedded deployment due to their greater computational and memory requirements, despite their slightly better accuracy. When quantized and tuned, the LSTM architecture preserves temporal feature learning for SoC estimation within the micro controller's RAM, Flash, and processor

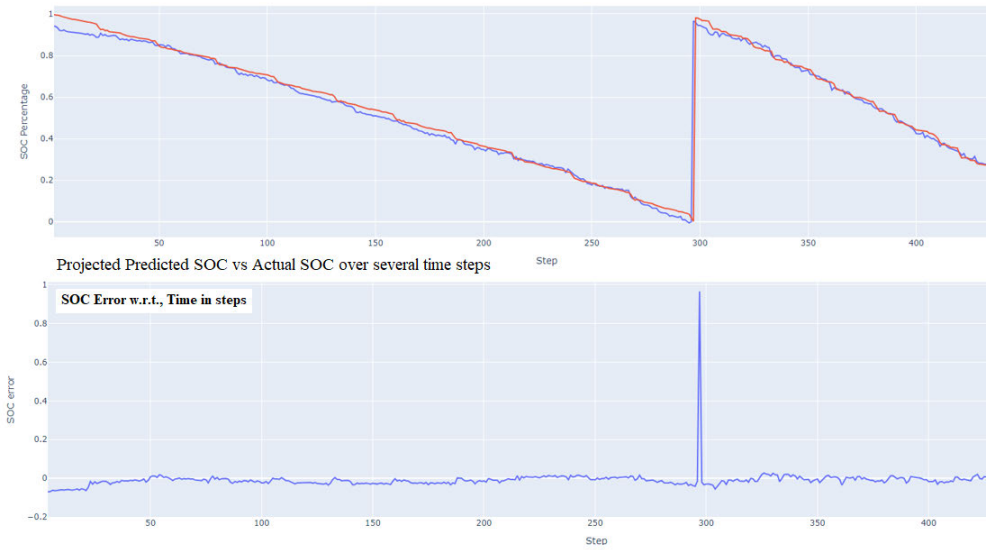


**FIGURE 12.** (TOP)Time-series comparison between predicted and true SoC values over a discharge cycle, (BOTTOM) Plot represents SoC error for time in steps.

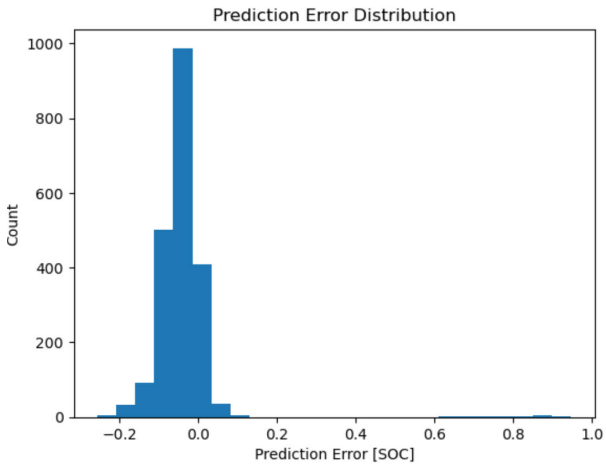


**FIGURE 13.** Histogram showing prediction error distribution for the proposed LSTM model.

**TABLE 3.** Comparing SoC estimation methods' MAE, RMSE, and MAXE percentages and illustrating the quantized LSTM model's better edge deployment trade-off.

| | METHOD | MAE(%) | RMSE(%) | MAXE(%) |
|---|---|---|---|---|
| [27] | GRU-RNN | 0.5 | – | 2.4 |
| [28] | PANN, BiLSTM | – | 0.0298 | – |
| [29] | ELSTM ASVDUKF | 0.0939 | 0.1074 | – |
| [30] | LSTM-RNN AEKF | 0.64 | 1.01 | 2.5 |
| [31] | EI-LSTM CO_LSTM | – | 1.3 | 3.2 |
| [32] | Ind LSTM | 0.6389 | 0.7633 | 2.2398 |
| [33] | UKF-LSTM | – | 2.3 | – |
| [34] | Ada Boost RT-LSTM | 1.54 | 2.15 | – |
| | Proposed LSTM model | **0.0630** | **0.1033** | – |

limits. Quantization balances computational complexity and estimation accuracy by reducing memory footprint without affecting predictive performance. For realistic, long-term SoC monitoring in edge-based battery-powered systems, low latency, power consumption, and reliability are more important than theoretical accuracy increases.

### E. MODEL QUANTIZATION STRATEGY

To enable efficient edge deployment, the trained LSTM model—initially stored in Keras `.h5` format—is converted to TensorFlow Lite (TFLite) format using dynamic range quantization. This step reduces the model's memory footprint by transforming the original 32-bit floating-point weights into 8-bit integers.

The quantization process compresses the original model size from 83 kB (float32) to 47 kB (int8), thereby reducing memory usage and accelerating inference. Post-quantization benchmarking revealed an average inference time of 3.2 ms per sample on the target microcontroller. Despite this compression, the model retained strong predictive performance, achieving a MAE of 0.0630, which closely matches the pre-quantization results.

This trade-off is considered acceptable in embedded contexts, where slight losses in accuracy are offset by substantial gains in latency and energy efficiency. The quantized model thus offers an effective balance between precision and resource constraints. For future work, Quantization-Aware Training (QAT) could be adopted to further minimize any degradation in accuracy introduced during the post-training quantization process.

### F. DEPLOYMENT ON MICROCONTROLLER

The quantized TensorFlow Lite model must be converted into a C-style array for integration into the embedded firmware. This is accomplished using the `xxd` utility, which generates

a header file that contains the model in hexadecimal byte format.

To deploy the model:

1) Train and export the model in `.h5` Keras format.
2) Convert it to a quantized TFLite model using TensorFlow Lite Converter with dynamic range quantization enabled.
3) Use `xxd` to convert the `.tflite` file into a C array (`model.cc/h`).
4) Integrate the TensorFlow Lite for Microcontrollers (TFLM) library into the embedded development environment (e.g., Arduino IDE or PlatformIO).
5) Include the generated `model.cc` and initialize the model in the inference code.
6) Compile and flash the firmware onto the target device.

This workflow ensures real-time inference is executed directly on-device without relying on any external cloud services, satisfying the stringent requirements of battery-powered systems. The final embedded deployment is both memory-efficient and latency-optimized, making it suitable for low-power applications such as EVs, drones, or energy storage systems. Liu et al. [39] provide a detailed survey of lightweight deep learning methods for deployment in resource-constrained environments such as edge devices and IoT systems. They classify key techniques including model compression, neural architecture search, quantization, pruning, and knowledge distillation. The study also explores emerging trends in adapting Large Language Models (LLMs) for low-resource hardware. Major challenges highlighted include maintaining accuracy, reducing latency, and improving energy efficiency. Li et al. [40] propose a Physics-Informed Mamba Neural Network (PIMNN) that integrates electrochemical physical laws into a deep learning framework via differential equations to improve SoC estimation for LIB. By modeling dynamic, transient, and quasi-steady states using potential data and circuit characteristics such as internal resistance and open-circuit voltage, their approach enhances both the interpretability and accuracy of long-term SoC prediction.

## IV. DATA ACQUISITION AND SYSTEM DESIGN

### A. BATTERY PROFILING AND DATA ACQUISITION

The SoC estimation process can be described in the following steps.

#### 1) DETAILS OF THE 18650 Li-ION BATTERY

In the realm of energy storage and battery management, precise data acquisition is paramount to ensure the efficient and safe operation of Li-ion batteries. Table:4 gives voltage standards of different Li-ion chemistries available in the market. This comprehensive procedure outlines the detailed steps for acquiring crucial data, including voltage, current, and temperature measurements from an 18650 Li-ion battery. These measurements are vital for monitoring the battery's performance, assessing its SoC, and optimizing its utilization in various temperature conditions.

**TABLE 4. Rechargeable battery chemistries.**

| Cell Chemistry | | Lithium Cobalt Oxide | Lithium Ferrous Phosphate | Lithium Nickel Cobalt Aluminium Oxide | Lithium Nickel Manganese Cobalt Oxide |
|---|---|---|---|---|---|
| Name | | LCO | LFP | NCA | NMC |
| Electrodes | Anode | Graphite | | | |
| | Cathode | LiCoO2 | LiFePO4 | LiNiCo AlO2 | LiNiMn* CoO2 * |
| Voltage | CutOff | 2.5 | 2.0 | 3.0 | 2.50 |
| | Nominal | 3.7 | 3.2 | 3.6 | 3.78 |
| | Vmax | 4.2 | 3.6 | 4.3 | 4.35 |

* NMC : INR 18650-30Q

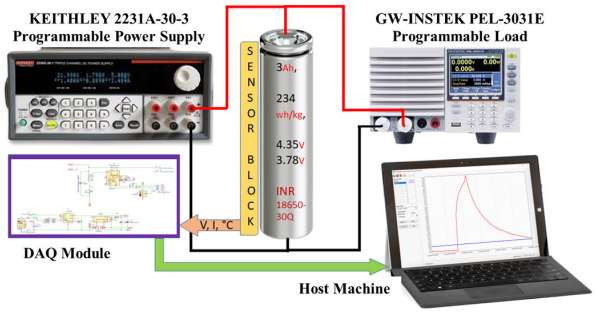


**FIGURE 14. Real-time laboratory setup used for battery data collection for edge deployment validation, including power supplies, loads, and DAQ sensing circuits.**

#### 2) EXPERIMENTAL SETUP AND FRAMEWORK

The experimental setup for estimating SoC involves mainly two methods: Laboratory Framework For Acquiring Battery Parameters and A Real-time approach for estimating SoC.

**Laboratory Framework For Acquiring Battery Parameters:**

Estimating the SoC of a 3Ah-18650 lithium-ion battery necessitates multiple processes and demands precise instruments and methodologies to guarantee accuracy and safety. The configuration is illustrated in Fig. 14 The equipment and quality instruments can provide a comprehensive and meticulous assessment of the battery's condition. The items employed for this purpose are enumerated below. Figure 15 illustrates the single-cell battery under test (BUT) within a temperature-regulated chamber. The host PC monitors and records battery parameters.

1) Programmable Load:GW-INSTEK PEL-3031E.
2) Programmable Power Supply: KEITHLEY 2231A-30.
3) Data Acquisition Module.

Battery Conditioning: If the battery is not new, perform a few charge-discharge cycles to ensure stability. This typically involves charging the battery to its maximum voltage and then discharging it to its cut-off voltage. Initial Setup: Connect the 18650 battery to the programmable load (GW-INSTEK PEL-3031E) and programmable power supply (KEITHLEY 2231A-30-3) in a circuit that allows both charging and discharging. The 'Data Acquisition Module' should be connected for monitoring and recording the voltage, current, and temperature during the test.

#### 3) CALIBRATION OF MEASUREMENT EQUIPMENT

Programmable Power Supply (KEITHLEY 2231A-30-3): Set up the power supply to the charging voltage recommended

**FIGURE 15.** Temperature-controlled environmental chamber used for testing SoC estimation at various thermal conditions ranging from 0°C to 45°C.

by the battery manufacturer, typically around 4.2 V for a Li-ion cell, and limit the charging current to a safe level, such as 1C (3 A for a 3 Ah battery). Programmable Load (GW-INSTEK PEL-3031E): Configure the load to discharge the battery at a specific rate. An ordinary rate for testing might be 0.5C or 1C. Ensure that the discharge cut-off voltage is configured in accordance with the battery's specifications, typically approximately 2.7 V. Data Acquisition System: Set up the DAQ to record voltage, current, and temperature at a suitable sampling rate to accurately capture the battery's performance throughout the test cycle.

#### 4) SoC ESTIMATION PROCEDURE

Charge Phase: Begin by fully charging the battery using the power supply while monitoring the process with the DAQ Module.Charging should be done according to the CC-CV method, where the battery is charged at a constant current till it reaches its maximum voltage. It is then charged at a constant voltage till the draining current drops to a low cut-off level. Discharge Phase: After the battery is fully charged, switch to the programmable load to begin discharging the battery at the predetermined rate. Monitor and record the discharge process with the DAQ module. Data Analysis: Using the data collected by the DAQ module, analyze the voltage, current, and temperature profiles to estimate the SoC. The SoC can be estimated based on the amount of energy discharged from the battery, comparing it to its rated capacity (3Ah in this case). The relationship between voltage and SoC will be specific to the battery chemistry and must be determined from the manufacturer's data or through empirical testing.

#### 5) PREPROCESSING AND FINAL STEPS

Repeat the charge-discharge cycle if multiple SoC points are needed to create a detailed SoC curve. Analyze the data collected to determine the battery's efficiency, capacity, and potential degradation over the test cycles.

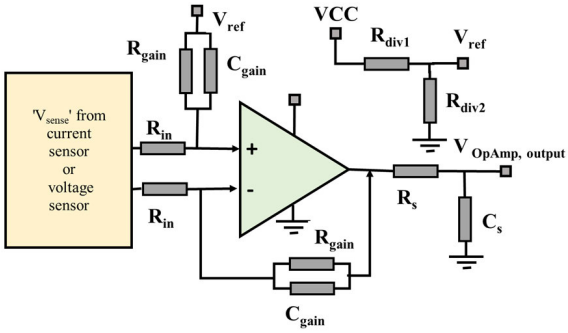


**FIGURE 16.** Generic schematic diagram illustrating the voltage and current sensing network used for signal conditioning.

### B. EDGE HARDWARE SYSTEM: DESIGN AND BLOCK DIAGRAM

**A Real-time approach for estimating SoC:** The overall block diagram is shown in Fig. 1 and the generic schematic for voltage and current sensing is shown in Fig. 16. primarily focuses on current sensing and signal conditioning, essential for monitoring current flow in systems such as those powered by batteries. It operates by first measuring the current via a "sense resistor" $R_{sense}$, where the voltage drop across it $V_{sense}$ is directly proportional to the current due to Ohm's law. Table 5 and design equations shows how $V_{sense}$ is calculated. To facilitate accurate and noise-free signal processing, a reference voltage $V_{ref}$ of 1.65 V is generated from the main supply voltage $V_{cc}$, 3.3 V through a voltage divider made up of resistors $R_{div1}$ and $R_{div2}$. This reference voltage sets the midpoint of the output range, ensuring that the output remains unipolar and above zero, enhancing the signal-to-noise ratio. The operational amplifier (X1), an AD8609, is configured in a non-inverting amplifier layout, where resistors $R_{gain}$ provide feedback for basic gain settings and $R_{in}$ is the input resistor. The effective gain should consider the entire network of feedback and connected resistors. Signal conditioning is further enhanced by including a low-pass filter at the output, formed by resistor $R_{gain}$ and capacitors $C_{gain}$ and additional RC network is provided in the output of the network,Rs and Cs which smoothens the output signal and remove high-frequency spikes(if any) before the signal reaches the microcontroller's ADC. Overall, this design translates the current through $R_{sense}$ into a voltage within a 0 to 3.3 V, which is the dynamic range of the Microcontroller. the signal is then digitally processed by the ADC, thereby enabling digital systems to monitor and manage current usage efficiently with high accuracy and stability.

#### 1) OP-AMP SELECTION

The following parameters are considered critical for selecting the operational amplifier.

1) Input offset voltage: The input offset voltage is critical for the design considering the accuracy to be achieved at the system level. A low input offset voltage makes sure that the error at Op-Amp input is minimal and

is minimally amplifies by the gain of the Op-Amp stage.

2) Operating temperature: The operating temperature of the present design can vary from $-5°C$ to $50°C$. The selected OpAmp operating temperature is from $-40°C$ to $+125°C$.
3) Supply voltage: The supply voltage of the Op-Amp ranges from 1.8 V to 5 V (min to max). Since the Vcc of Microcontroller is 3.3 V and it is in the range of Op-Amp power supply. Hence this Op-Amp is chosen.

The Op-Amp is operated in a differential mode. The voltage across '$R_{sense}$' is not with reference to the board ground. It is a differential voltage '$V_{sense}$' across the sense resistor, which is connected in series with the actual load. In the simulation, the load resistor '$R_{load}$' is considered to be 1.1 Ω since the sum of $R_{load}$ and Rsense becomes 1.2 Ω, which essentially accounts for a 3 A load current with a 3.6 V rated battery. The selected Li-ion battery has a rated current of 3 A. Hence the $R_{load}$ is accordingly considered.

#### 2) REFERENCE VOLTAGE (VREF)

The voltage reference $V_{ref}$ 1.65 V is generated by a potential divider network connected across $V_{cc}$ 3.3 V and ground.

#### 3) GAIN STAGE DESIGN

The gain of the Op-Amp stage circuit is 3.3. This gain is selected so as to utilize the available dynamic range of Op-Amp output in an efficient way. Given that $R_{input} = 10\,\text{k}\Omega$ and $R_{gain} = 33\,\text{k}\Omega$, As the battery voltage varies from 3.2V to 4.2V, the Op-Amp output is designed to vary from 3V to 3.25V.

#### 4) DESIGN EQUATIONS

$$V_{\text{ref}} = \frac{V_{cc} \cdot R_{div1}}{R_{div1} + R_{div2}} \tag{14}$$

$$\text{Gain}\{A\} = \frac{R_{\text{gain}}}{R_{\text{input}}} \tag{15}$$

$$V_{\text{input,Offset,Temp}} = (\text{Temp}_{\text{HI}} - \text{Temp}_{\text{LO}}) \left(\frac{dV_{\text{inputOffset}}}{dT}\right) + V_{\text{inputOffset}} \tag{16}$$

$$V_{\text{Op-Amp,Output,Temp}} = (V_{\text{sense}} + V_{\text{input,Offset,Temp}}) \times \text{Gain} + V_{\text{ref}} \tag{17}$$

$$\text{ACE} = \left(\frac{V_{\text{Op-Amp,Output,min}} - V_{\text{Op-Amp,Output,Typ}}}{V_{\text{Op-Amp,Output,Typ}}}\right) \times 100 \tag{18}$$

$$\text{ACE Temp} = \frac{V_{\text{Op-Amp,output,temp,min}} - V_{\text{Op-Amp,output,temp,typ}}}{V_{\text{Op-Amp,output,temp,typ}}} \times 100$$

$$\text{ACE : Analog Circuit Error} \tag{19}$$

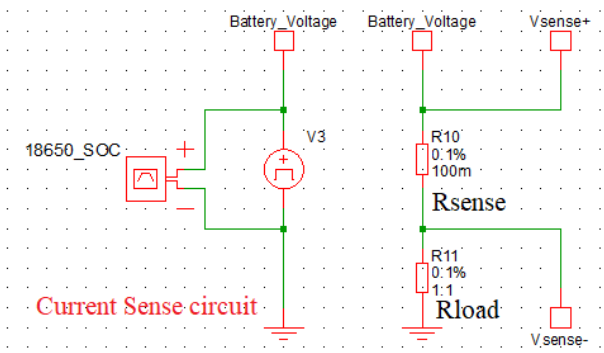


**FIGURE 17.** Hardware implementation of the current sensing circuit schematic using a precision shunt resistor.

**TABLE 5.** Calculations for the Current Sense Circuit.

| 0.1 R tolerance | Tolerance | Min | Typ | Max |
|---|---|---|---|---|
| **Sense Resistor and Load Current** | | | | |
| $V_{battery}$ | 0 | 3.6 | 3.6 | 3.6 |
| $R_{load}$ | 0 | 1.1 | 1.1 | 1.1 |
| $R_{sense}$ | 0.10% | 0.0999 | 0.1 | 0.1001 |
| $R_{load,eff}$ | | 1.1999 | 1.2 | 1.2001 |
| $I_{load,eff}$ | | 2.99975 | 3 | 3.00025 |
| $V_{sense}$ | | 0.299675 | 0.3 | 0.300325 |
| $V_{ref}circuit(FortheDCoffset)$ | | | | |
| $V_{cc}$ | 0 | 3.3 | 3.3 | 3.3 |
| $R_8$ | 0.10% | 99.9 | 100 | 100.1 |
| $R_7$ | 0.10% | 99.9 | 100 | 100.1 |
| $V_{ref}$ | | 1.64835 | 1.65 | 1.65165 |
| **Op-Amp gain stage** | | | | |
| $R_{input}$ | 0.1% | 9990 | 10000 | 10010 |
| $R_{gain}$ | 0.1% | 32967 | 33000 | 33033 |
| Gain | | 3.293 | 3.3 | 3.306 |
| $V_{in,Op-Amp}$ | | 0.299 | 0.3 | 0.300 |
| $V_{in,offset}(mV)$ | | 0 | 0.012 | 0.05 |
| $V_{Op-Amp,output}$ | | 2.6353 | 2.6400 | 2.6449 |
| $dV_{input,offset}/dT$ | | 4.5E-06 | 4.5E-06 | 4.5E-06 |
| operating;Temp | | -10 | 25 | 100 |
| $V_{input,offset,temp}(mV)$ | | 0.158 | 0.012 | 0.388 |
| $V_{Op-Amp,output,temp}$ | | 2.6348 | 2.6400 | 2.6460 |
| **Analog circuit error** | | -0.1795 | 0.0000 | 0.1830 |
| Analog circuit error,temp | | -0.1991 | 0.0000 | 0.2253 |
| **ADC Error** | | | | |
| ADC resolution | | 12 | 12 | 12 |
| Number of steps | | 4096 | 4096 | 4096 |
| Step size (mV) | | 0.806 | 0.806 | 0.806 |
| Maxquantization error (mv) | | 0.403 | 0.403 | 0.0403 |
| Maxquantization error (%) | | 0.0122 | 0.0122 | 0.0122 |
| **Overall Circuit error (%)** | | **-0.211** | **0.012** | **0.238** |

#### 5) ADC CONFIGURATION AND RESOLUTION

A 12-bit and a 16-bit ADC resolution are considered in the design calculations. Based on the optimum percentage error,an ADC with 12-bit resolution is chosen.

### C. CURRENT SIGNAL CONDITIONING

#### 1) CIRCUIT SCHEMATIC

The generic Op-Amp-based schematic is shown in Figure 16 is used for amplifying the voltage equivalent of the sensed current. The conversion of sensed current into its equivalent voltage is shown in Fig. 17

#### 2) DESIGN CALCULATIONS FOR CURRENT SENSING

Design calculations are shown in the Table 5. The output voltage at every stage of the circuit is calculated based on the component tolerances. The overall worst-case percentage

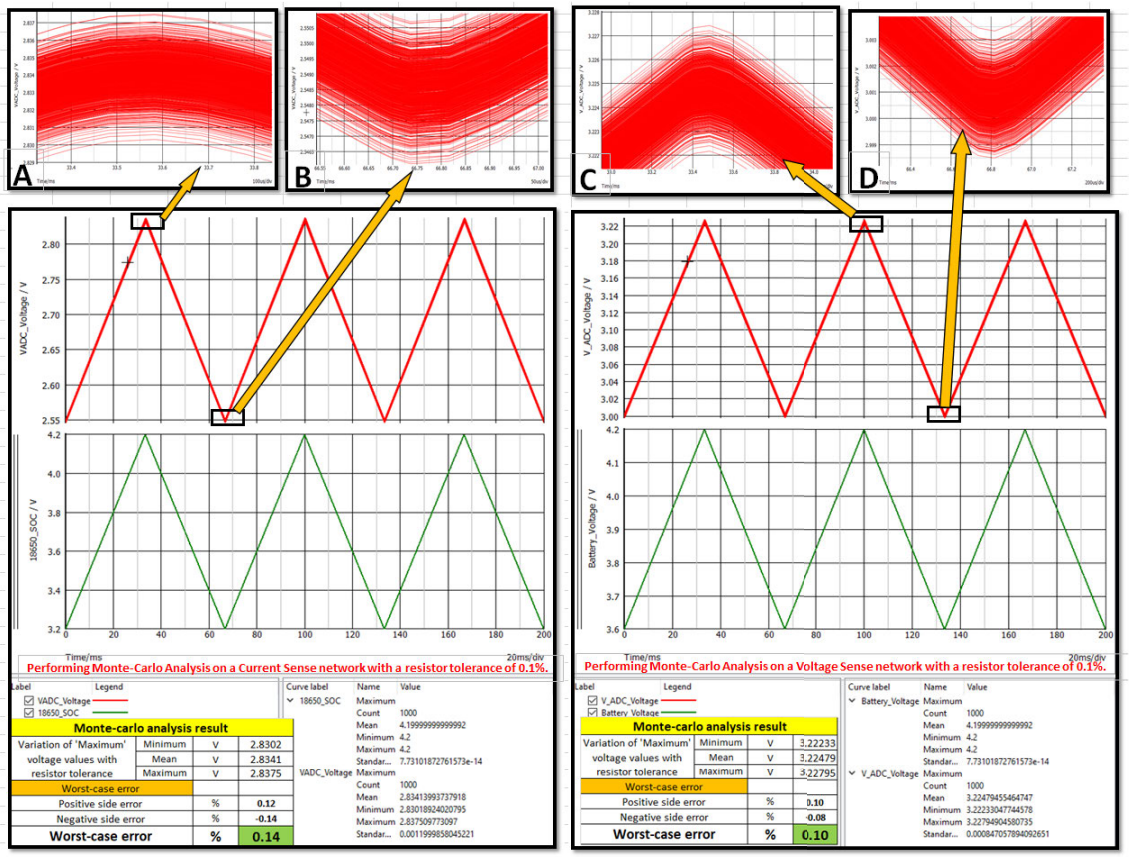


**FIGURE 18.** Monte Carlo simulation results of the current and voltage sensing networks, considering resistor tolerances of ±0.1%. A & B (TOP-Left): shows the boundary conditions current sensing circuit, C & D (TOP-Right): shows the boundary conditions voltage sensing circuit, while running Monte Carlo analysis for 1 k runs.

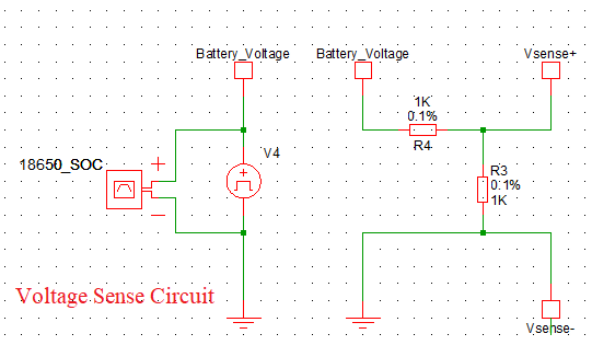


**FIGURE 19.** Voltage Sense circuit schematic used to scale down the battery terminal voltage for microcontroller-safe input levels.

error at the ADC of the Microcontroller is then calculated considering the ADC resolution.

Table 5 shows the design calculations for resistor tolerance of 0.1% for a Voltage sense circuit. Other tolerances like 10% and 1% were also considered and discarded because of high overall worst-case error (Not shown in the table).

### 3) MONTE CARLO ERROR ANALYSIS FOR CURRENT SENSING

Monte Carlo analysis is a statistical technique that uses random sampling to understand how they influence the risk and uncertainty involved in quantitative prediction and forecasting models. This approach assigns random values for input parameters according to defined probability distributions, simulates outcomes for each set of values, and performs this operation a large number of times, resulting in a distribution of possible results. By further applying this method on to our data in our example, which contains parameters with tolerances that allow for uncertainties that arise in values of resistors and voltages and other electrical components, we can gain valuable observations called Monte Carlo analysis. The left side part of Fig. 18 shows the analysis of the current sense circuit for a given R value as 0. 1%.Suppose that we have $R_{sense}$, $R_{load}$ and $V_{battery}$ values with their corresponding tolerance to be represented by a probability distribution. These parameters are used to generate random samples, in case of each set of samples, dependent variables like $V_{sense}$, $V_{Op-Amp,output}$ are calculated using the given circuit equations. Standard distributions are calculated over the output from many simulations to gauge the variability and likelihood of various outcomes. This assists in measuring the risks and uncertainties the system its performance. This can be exemplified by Monte Carlo analysis on the $V_{Op-Amp,output}$ parameter, where random $R_{Sense}, R_{Load}, V_{Battery}, R_{gain}, R_{in} \& R_s$ samples are generated for each iteration, $V_{Op-Amp,output}$ is calculated for every

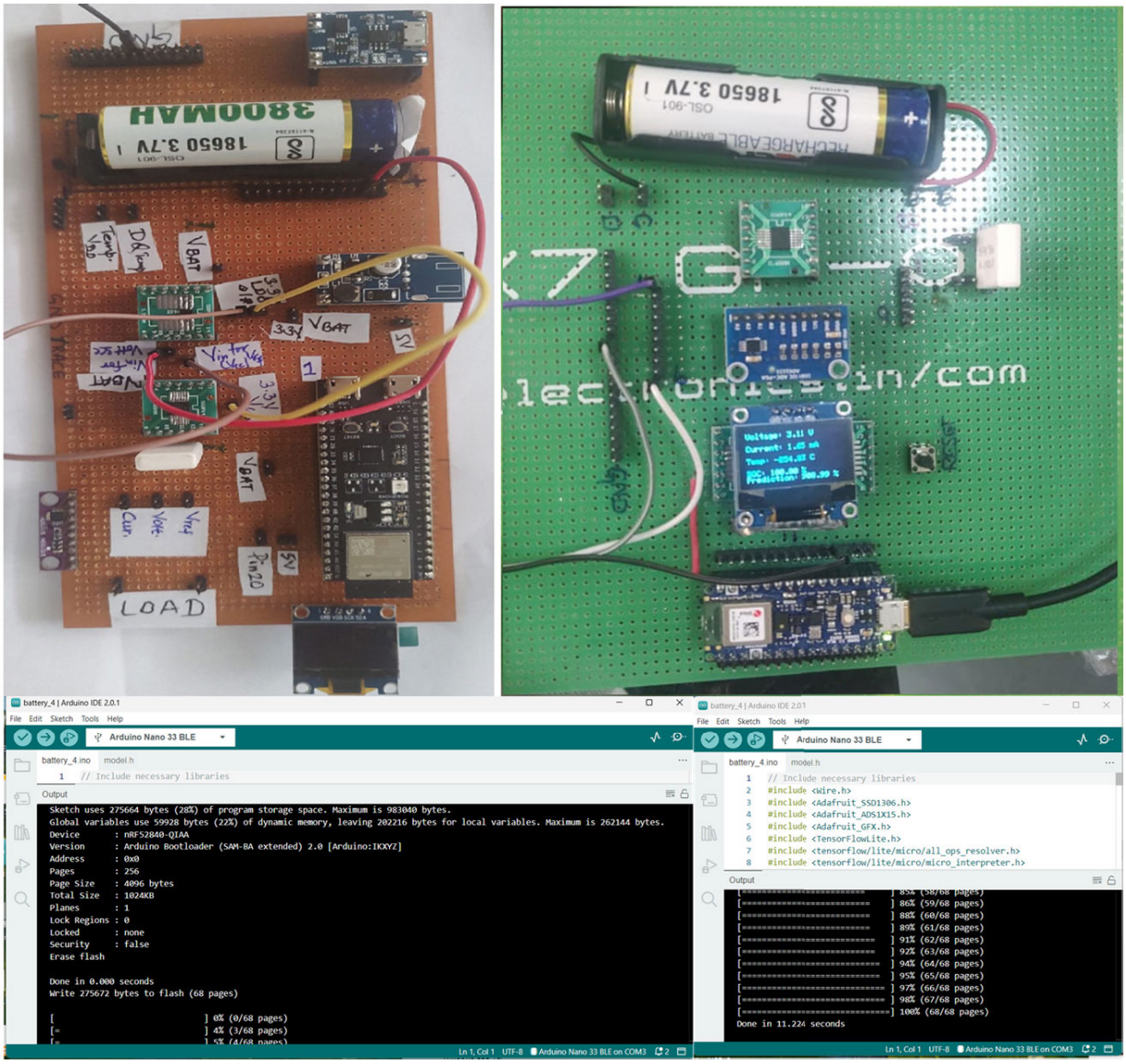


**FIGURE 20.** (TOP) Edge device prototypes with the deployed LSTM model. Left: ESP32-S3, Right: Arduino Nano 33 BLE Sense. (BOTTOM) Deployment using Arduino IDE showing model upload interface (left) and deployment time tracking (right).

wandering iteration, and the statistical distribution is examined. It provides visibility into both the distribution, as well as the outliers of $V_{Op-Amp,output}$ and thus helps engineers assess the likelihood of various scenarios and make more knowledgeable choices. So it presents a complete view of how Monte Carlo findings on input variability can prevent undetermined results.The lower left corner of the figure shows the table, which gives the boundary conditions. The worst case error is 0.14% from the calculations. Fig.18: A&B, shows the boundary conditions while running Monte Carlo analysis for 1k runs.

### D. VOLTAGE SIGNAL CONDITIONING

#### 1) CIRCUIT SCHEMATIC

The schematic diagram of the voltage sense circuit using AD8609 is shown in Fig. 19

#### 2) DESIGN CALCULATIONS FOR VOLTAGE SENSING

The design calculations for the voltage sense circuit is shown in Table 6. The output voltage at every stage of the circuit is calculated based on the component tolerances. The overall worst-case percentage error at the ADC of the microcontroller is then calculated, considering the ADC resolution.

#### 3) MONTE CARLO ERROR ANALYSIS FOR VOLTAGE SENSING

Monte-Carlo analysis for the voltage sensing circuit is also done in the same way as the current sense and the results are shown in the right side part of Fig. 18 and Fig. 18: C&D.

### E. TEMPERATURE SENSING AND COMPENSATION

**Temperature Measurement:** Temperature plays a crucial role in battery performance and safety. The temperature

**TABLE 6.** Calculations for the voltage sense circuit.

| 0.1 R Tolerance | Tolerance | Min | Typ | Max |
|---|---|---|---|---|
| **Sense Resistor and Load Current** | | | | |
| $V_{battery}$ | 0 | 4.2 | 4.2 | 4.2 |
| $R_1$ | 0.1% | 999 | 1000 | 1001 |
| $R_2$ | 0.10% | 999 | 1000 | 1001 |
| $V_{sense}$ | | 2.0979 | 2.1 | 2.1021 |
| $V_{refCircuit}(Forthe DCoffset)$ | | | | |
| $V_{cc}$ | 0 | 3.3 | 3.3 | 3.3 |
| $R_8$ | 0.1% | 99.9 | 100 | 100.1 |
| $R_7$ | 0.1% | 99.9 | 100 | 100.1 |
| $V_{ref}$ | | 1.64835 | 1.65 | 1.65165 |
| **Op-Amp gain stage** | | | | |
| $R_{input}$ | 0.1% | 99900 | 100000 | 100100 |
| $R_{gain}$ | 0.1% | 74925 | 75000 | 75075 |
| Gain | | 0.748501 | 0.75 | 0.751502 |
| $V_{in,Op-Amp}$ | | 2.0979 | 2.1 | 2.1021 |
| $V_{input,offset}$ | | 0 | 0.000012 | 0.00005 |
| $V_{Op-Amp,output}$ | | 3.2186 | 3.2250 | 3.2314 |
| $dV_{input,offset}/dT$ | | 4.5E-06 | 4.5E-06 | 4.5E-06 |
| Operating;Temperature | | -10 | 25 | 100 |
| $V_{input,offset,temp}(mV)$ | | 0.158 | 0.012 | 0.388 |
| $V_{Op-Amp,output,temp}$ | | 3.2185 | 3.2250 | 3.2317 |
| **Analog Circuit Error** | | **-0.1978** | **0.0000** | **0.1988** |
| Analog Circuit Error,temp | | -0.2014 | 0.0000 | 0.2066 |
| **ADC Error** | | | | |
| ADC resolution | | 12 | 12 | 12 |
| Number of steps | | 4096 | 4096 | 4096 |
| Step size (mV) | | 0.806 | 0.806 | 0.806 |
| Max quantization error (mV) | | 0.403 | 0.403 | 0.403 |
| Max quantization error (%) | | 0.0122 | 0.0122 | 0.0122 |
| **Overall Circuit error (%)** | | **-0.214** | **0.012** | **0.219** |

**TABLE 7.** Resistor tolerance and ADC resolution.

| Tolerance in % | | | | | | | ADC Resolution | Overall Worst Case Error | |
|---|---|---|---|---|---|---|---|---|---|
| R sense | R in1 | R in2 | R gain1 | R gain2 | R div1 | R div2 | | Current Sense | Voltage Sense |
| 10 | 10 | 10 | 10 | 10 | 10 | 10 | 12 | 14.703 | 21.961 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 12 | 1.484 | 2.018 |
| **0.1** | **0.1** | **0.1** | **0.1** | **0.1** | **0.1** | **0.1** | **12** | **0.238** | **0.219** |
| 10 | 10 | 10 | 10 | 10 | 10 | 10 | 16 | 14.691 | 21.949 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 16 | 1.473 | 2.006 |
| 0.1 | 0.1 | 0.1 | 0.1 | 0.1 | 0.1 | 0.1 | 16 | 0.226 | 0.207 |

sensor used for this application is DS18B20. The DS18B20 is a digital thermometer that provides the digital equivalent of its analog reading in terms of 9-bit to 12-bit resolution in Celsius format. The sensor, along with temperature measurement, gives a feature alarm function with upper and lower trigger points with which one can be programmed inside the non-volatile memory with the user-programmable. The DS18B20 communicates over a 1-Wire bus - by definition, requires only one data line (and ground) for communication with a host edge device. DS18B20 is connected to a microcontroller's digital input pin.

### *F. FINAL INFERENCE ARCHITECTURE AND DISCUSSION*

This study involved the development and implementation of a predictive model to estimate the SoC of a battery. The model uses voltage, current, and temperature as input parameters. The experimental setup included a climatic chamber for accurate environmental testing and a microcontroller-based edge device for real-time data acquisition and analysis. The implemented current and voltage sensing circuits ensures accurate measurements that are essential for precise SoC predictions. We successfully deployed an LSTM model on a microcontroller with minimal memory usage by training and publishing it in different formats such as Keras and TensorFlow Lite. The size of the quantized model was significantly decreased to 47 KB, allowing it to be deployed in applications with limited resources. The comprehensive system, comprising the edge device and sensing circuits, exhibited successful real-time SoC prediction, emphasising the feasibility of employing machine learning for battery management in Internet of Things (IoT) applications. This work establishes a foundation for future enhancements and incorporation of sophisticated predictive models in intelligent energy systems. Fig. 20 shows the authors' prototypes where an ESP32 S3 platform on the left and the Arduino Nano 33 BLE Sense on the right are used as an edge device where the SoC and different battery parameters are displayed. Based on the design calculations for Monte Carlo analysis on voltage and current sensing circuits, it is evident that a resistor tolerance of 0.1% and an ADC resolution of 12-bit are the most suitable choices to keep the total worst-case error within acceptable ranges. Table 7 displays the ultimate criteria for the design.

#### 1) DEVICE-LEVEL BENCHMARKING

To further validate edge performance, a comprehensive comparison was conducted using two edge platforms—Arduino Nano 33 BLE and ESP32-S3-WROOM-1—based on model footprint, inference time, memory usage, and energy consumption. The quantized LSTM model attained inference times of roughly 3.2 ms on the Arduino and 2.5 ms on the ESP32, considerably surpassing previous studies and commercial alternatives such as Edge Impulse. Memory and flash usage remained within 28% and 22% on the Arduino, while ESP32 utilized just 6% flash and 9% RAM. Energy consumption per inference was calculated using a USB power analyzer by measuring current draw during inference and applying $E = V \times I \times t$, resulting in 21.12 mJ (Arduino) and 16.8 mJ (ESP32). Full metrics are shown in Table 9, confirming the ESP32's higher efficiency and available memory headroom (467 KB vs. 202 KB), further supporting its suitability for advanced deployments.

The SoC estimate methodology on the edge device was compared to a cloud-based technique to demonstrate its benefits. Performance parameters included inference delay and energy usage.

#### 2) ANALYSIS OF LATENCY

The edge-based model infers predictions in 45 milliseconds on average. The average forecast delay for cloud-based estimate is 320 milliseconds due to wireless data transfer to a remote server and response. Edge deployment decreases latency by roughly 85%, allowing real-time SoC updates for integrated battery management.

#### 3) ENERGY CONSUMPTION ANALYSIS

The edge device uses around 82.5 mW of power during inference. At 45 ms inference, each prediction consumes 3.71 mJ of energy. Cloud-based inference uses 15–20 mJ

**TABLE 8.** Comparative analysis of model deployments.

| Metric | Proposed Model | Recent Similar Work [37] | Edge Impulse, [38] |
|---|---|---|---|
| Inference Time per Sample | ~3.2 ms | ~130 ms | ~17 ms |
| Flash Memory Usage | 120 KB | 186.3 KB | 55 KB |
| RAM Usage | 18 KB | 29.1 KB | 10 KB |
| Model Size | 47 KB | 169.3 KB | 50 KB |
| Quantization | Yes (int8) | Yes | Yes |
| Energy used / Inference | ~21.12 mJ | ~858 mJ | ~112.2 mJ |

Note: Energy (in joules)≈Voltage (3.3V)×Current (2A)×Time (s)

**TABLE 9.** Comparison of deployment metrics on Arduino Nano 33 BLE and ESP32-S3-WROOM-1.

| Aspect | Arduino Nano 33 BLE | ESP32-S3-WROOM-1 |
|---|---|---|
| Target Device | nRF52840-QIAA @ 64 MHz | Xtensa LX7 Dual-core @ 240 MHz |
| Program Storage Usage | 275,664 bytes (28% of 983,040 bytes) | ~190,000 bytes (6% of 4 MB flash) |
| Flash Memory | 1 MB (internal) | 4 MB (external QSPI) |
| Dynamic Memory Usage | 59,928 bytes (22% of 256 KB) | ~45,000 bytes (9% of 512 KB) |
| Available Memory | ~202,216 bytes | ~467,000 bytes |
| Deployment Time | 11.224 seconds | ~6.8 seconds |
| Pages Written | 68/68 (100%) | ~75 pages |
| Model Format | Quantized TFLite (C-array in `model.h`) | Quantized TFLite (C-array in `model.h`) |
| Key Libraries | TFLite Micro, ADS1X15, SSD1306, Wire | TFLite Micro, ADS1X15, SSD1306, Wire |
| Inference Time per Sample | ~3.2 ms | ~2.5 ms |
| Energy Consumption per Inference | ~21.12 mJ | ~16.8 mJ |
| Deployment Platform | Arduino IDE 2.0.1 | Arduino IDE 2.0.1 |

per prediction due to wireless transmission energy and longer device active durations. This shows that edge-based deployment saves 75–80% energy over cloud-based alternatives [36]. Table 8 compares this with a recent work [37] and an official toolchain: Edge Impulse [38].

## V. CONCLUSION

This paper provides a detailed analysis of the SoC estimate, including an overview of the fundamentals and an exploration of several data-driven methodologies used in the estimation of SoC over the last five years. Our research aims to comprehensively analyze battery datasets, specifically examining the relationship between various aspects of the dataset and the stability of the dataset. These factors are essential for accurately assessing the effectiveness of LSTM models. In addition, we explored several battery chemistries and edge computing platforms, providing a detailed description of our experimental configuration for SoC estimation. We utilized Monte Carlo analysis in our method to design circuit components. Additionally, we employed suitable software frameworks, libraries, and assessment criteria. The findings of our trial showed that our procedures were effective, as indicated by a MAE of 0.0630, a MSE of 0.0107, and a RMSE of 0.1033. These measures demonstrate that our algorithm delivers precise SoC forecasts with low variance from the actual values. In conclusion, our contributions provide valuable information about SoC estimates. We propose comprehensive approaches for battery statistics analysis and effective LSTM model development. Our study also highlights how crucial it is to use Monte Carlo analysis to assist the design equation approach for circuit design and edge computing platforms in order to create trustworthy SoC estimate systems. Our findings can be used in future research to improve the accuracy and reliability of SoC estimates in a range of applications.

## REFERENCES


[1] (Nov. 12, 2023). *UN Climate Change Conference—United Arab Emirates*. [Online]. Available: https://unfccc.int/news/cop28-agreement-signals-beginning-of-the-end-of-the-fossil-fuel-era

[2] J. F. Peters, M. Baumann, B. Zimmermann, J. Braun, and M. Weil, "The environmental impact of Li-ion batteries and the role of key parameters—A review," *Renew. Sustain. Energy Rev.*, vol. 67, pp. 491–506, Jan. 2017, doi: 10.1016/j.rser.2016.08.039.

[3] P. Shrivastava, T. K. Soon, M. Y. I. B. Idris, and S. Mekhilef, "Overview of model-based online state-of-charge estimation using Kalman filter family for lithium-ion batteries," *Renew. Sustain. Energy Rev.*, vol. 113, Oct. 2019, Art. no. 109233, doi: 10.1016/j.rser.2019.06.040.

[4] C.-X. Yu, Y.-M. Xie, Z.-Y. Sang, S.-Y. Yang, and R. Huang, "State-of-charge estimation for lithium-ion battery using improved DUKF based on state-parameter separation," *Energies*, vol. 12, no. 21, p. 4036, Oct. 2019, doi: 10.3390/en12214036.

[5] D. Sesidhar, C. Badachi, and R. C. Green, "A review on data-driven SOC estimation with Li-ion batteries: Implementation methods & future aspirations," *J. Energy Storage*, vol. 72, Nov. 2023, Art. no. 108420, doi: 10.1016/j.est.2023.108420.

[6] M. Zhang and X. Fan, "Review on the state of charge estimation methods for electric vehicle battery," *World Electr. Vehicle J.*, vol. 11, no. 1, p. 23, Mar. 2020, doi: 10.3390/wevj11010023.

[7] X. Liu, L. Chen, L. Zhu, J. Wang, L. Chen, X. Zeng, Z. Song, and L. Wang, "High-accuracy battery state of charge estimation strategy based on deep belief network cascaded with extended Kalman filter," *J. Electrochem. Energy Convers. Storage*, vol. 21, no. 3, Aug. 2024, Art. no. 031006, doi: 10.1115/1.4063431.

[8] C. G. Manriquez-Padilla, I. Cueva-Perez, A. Dominguez-Gonzalez, D. A. Elvira-Ortiz, A. Perez-Cruz, and J. J. Saucedo-Dorantes, "State of charge estimation model based on genetic algorithms and multivariate linear regression with applications in electric vehicles," *Sensors*, vol. 23, no. 6, p. 2924, Mar. 2023, doi: 10.3390/s23062924.

[9] C. Chen, Q. Zhang, W. Liao, F. Zhu, M. Li, and H. Wu, "State of charge estimation for lithium-ion batteries using an adaptive cubature Kalman filter based on improved generalized minimum error entropy criterion," *Green Energy Intell. Transp.*, vol. 5, Mar. 2025, Art. no. 100292, doi: 10.1016/j.geits.2025.100292.

[10] S. Schwunk, N. Armbruster, S. Straub, J. Kehl, and M. Vetter, "Particle filter for state of charge and state of health estimation for lithium–iron phosphate batteries," *J. Power Sources*, vol. 239, pp. 705–710, Oct. 2013, doi: 10.1016/j.jpowsour.2012.10.058.

[11] Á. Á. Anón, P. J. G. Nieto, F. J. De Cos Juez, F. S. Lasheras, M. G. Vega, and M. N. R. Gutiérrez, "Battery state-of-charge estimator using the SVM technique," *Appl. Math. Model.*, vol. 37, no. 9, pp. 6244–6253, May 2013, doi: 10.1016/j.apm.2013.01.024.

[12] I. A. Ibrahim and T. Khatib, "A novel hybrid model for hourly global solar radiation prediction using random forests technique and firefly algorithm," *Energy Convers. Manage.*, vol. 138, pp. 413–425, Apr. 2017, doi: 10.1016/j.enconman.2017.02.006.

[13] G. O. Sahinoglu, M. Pajovic, Z. Sahinoglu, Y. Wang, P. V. Orlik, and T. Wada, "Battery state-of-charge estimation based on regular/recurrent Gaussian process regression," *IEEE Trans. Ind. Electron.*, vol. 65, no. 5, pp. 4311–4321, May 2018, doi: 10.1109/TIE.2017.2764869.

[14] J. Yu, B. Sun, S. Gao, Z. Wang, and E. Hou, "Remote correction analysis of SOC accuracy based on deep belief network," in *Proc. Chin. Autom. Congr. (CAC)*, Oct. 2017, pp. 6072–6077, doi: 10.1109/CAC.2017.8243871.

[15] V. Pizarro-Carmona, S. Castano-Solís, M. Cortés-Carmona, J. Fraile-Ardanuy, and D. Jiménez-Bermejo, "GA-based approach to optimize an equivalent electric circuit model of a Li-ion battery-pack," *Expert Syst. Appl.*, vol. 172, Jun. 2021, Art. no. 114647, doi: 10.1016/j.eswa.2021.114647.

[16] Y.-S. Cheng, "Identification of parameters for equivalent circuit model of Li-ion battery cell with population based optimization algorithms," *Ain Shams Eng. J.*, vol. 15, no. 3, Mar. 2024, Art. no. 102481, doi: 10.1016/j.asej.2023.102481.

[17] M. S. Hossain Lipu, M. A. Hannan, A. Hussain, M. H. Saad, A. Ayob, and M. N. Uddin, "Extreme learning machine model for state-of-charge estimation of lithium-ion battery using gravitational search algorithm," *IEEE Trans. Ind. Appl.*, vol. 55, no. 4, pp. 4225–4234, Jul. 2019, doi: 10.1109/TIA.2019.2902532.

[18] W. Zheng, B. Xia, W. Wang, Y. Lai, M. Wang, and H. Wang, "State of charge estimation for power lithium-ion battery using a fuzzy logic sliding mode observer," *Energies*, vol. 12, no. 13, p. 2491, Jun. 2019, doi: 10.3390/en12132491.

[19] C. H. Cai, D. Du, and Z. Y. Liu, "Battery state-of-charge (SOC) estimation using adaptive neuro-fuzzy inference system (ANFIS)," in *Proc. 12th IEEE Int. Conf. Fuzzy Syst.*, vol. 2, St. Louis, MO, USA, May 2003, pp. 1068–1073, doi: 10.1109/FUZZ.2003.1206580.

[20] Z. Fu, B. Sun, J. Gong, M. Gong, X. Zhao, and S. Ma, "A SOC estimation method for Li-ion batteries under high-rate pulse conditions based on AO-BPNN model," *Space, Sci. Technol.*, vol. 3, p. 88, Jan. 2023, doi: 10.34133/space.0088.

[21] J. Du, Z. Liu, and Y. Wang, "State of charge estimation for Li-ion battery based on model from extreme learning machine," *Control Eng. Pract.*, vol. 26, pp. 11–19, May 2014, doi: 10.1016/j.conengprac.2013.12.014.

[22] Z. Xi, R. Wang, Y. Fu, and C. Mi, "Accurate and reliable state of charge estimation of lithium ion batteries using time-delayed recurrent neural networks through the identification of overexcited neurons," *Appl. Energy*, vol. 305, Jan. 2022, Art. no. 117962, doi: 10.1016/j.apenergy.2021.117962.

[23] C. Vidal, P. J. Kollmeyer, M. Naguib, P. Malysz, O. Gross, and A. Emadi, "Robust xEV battery state-of-charge estimator design using a feedforward deep neural network," *SAE Int. J. Adv. Current Practices Mobility*, vol. 2, no. 5, pp. 2872–2880, Apr. 2020, doi: 10.4271/2020-01-1181.

[24] Z. Zhang, Z. Dong, H. Lin, Z. He, M. Wang, Y. He, X. Gao, and M. Gao, "An improved bidirectional gated recurrent unit method for accurate state-of-charge estimation," *IEEE Access*, vol. 9, pp. 11252–11263, 2021, doi: 10.1109/ACCESS.2021.3049944.

[25] S. Bockrath, A. Rosskopf, S. Koffel, S. Waldh, K. Srivastava, and V. R. H. Lorentz, "State of charge estimation using recurrent neural networks with long short-term memory for lithium-ion batteries," in *Proc. 45th Annu. Conf. IEEE Ind. Electron. Soc.*, vol. 1, Lisbon, Portugal, Oct. 2019, pp. 2507–2511, doi: 10.1109/IECON.2019.8926815.

[26] P. Kollmeyer, C. Vidal, M. Naguib, and M. Skells, "G 18650HG2 Li-ion battery data and example deep neural network xEV SoC estimator script," Mendeley Data, V3, Tech. Rep., 2020, doi: 10.17632/cp3473x7xv.3.

[27] S. El Fallah, J. Kharbach, Z. Hammouch, A. Rezzouk, and M. O. Jamil, "State of charge estimation of an electric vehicle's battery using deep neural networks: Simulation and experimental results," *J. Energy Storage*, vol. 62, Jun. 2023, Art. no. 106904, doi: 10.1016/j.est.2023.106904.

[28] A. Manoharan, D. Sooriamoorthy, M. Begam, and V. R. Aparow, "Electric vehicle battery pack state of charge estimation using parallel artificial neural networks," *J. Energy Storage*, vol. 72, Nov. 2023, Art. no. 108333, doi: 10.1016/j.est.2023.108333.

[29] P. Takyi-Aninakwa, S. Wang, G. Liu, A. N. Bage, E. D. Bobobee, E. Appiah, and Q. Huang, "Enhanced extended-input LSTM with an adaptive singular value decomposition UKF for LIB SOC estimation using full-cycle current rate and temperature data," *Appl. Energy*, vol. 363, Jun. 2024, Art. no. 123056, doi: 10.1016/j.apenergy.2024.123056.

[30] H. Zhao, C. Liao, C. Zhang, L. Wang, and L. Wang, "State-of-charge estimation of lithium-ion battery: Joint long short-term memory network and adaptive extended Kalman filter online estimation algorithm," *J. Power Sources*, vol. 604, Jun. 2024, Art. no. 234451, doi: 10.1016/j.jpowsour.2024.234451.

[31] J. Chen, Y. Zhang, J. Wu, W. Cheng, and Q. Zhu, "SOC estimation for lithium-ion battery using the LSTM-RNN with extended input and constrained output," *Energy*, vol. 262, Jan. 2023, Art. no. 125375, doi: 10.1016/j.energy.2022.125375.

[32] P. Venugopal, T. Vigneswaran, and S. S. Reka, "State of charge estimation of lithium batteries in electric vehicles using IndRNN," *IETE J. Res.*, vol. 69, no. 5, pp. 2886–2896, Jul. 2023, doi: 10.1080/03772063.2021.1906770.

[33] Y. Zeng, Y. Li, and T. Yang, "State of charge estimation for lithium-ion battery based on unscented Kalman filter and long short-term memory neural network," *Batteries*, vol. 9, no. 7, p. 358, Jul. 2023, doi: 10.3390/batteries9070358.

[34] R. Li, P. Liu, K. Li, and X. Zhang, "AdaBoost.Rt-LSTM based joint SOC and SOH estimation method for retired batteries," *Batteries*, vol. 9, no. 8, p. 425, Aug. 2023, doi: 10.3390/batteries9080425.

[35] S. Tao, B. Jiang, X. Wei, and H. Dai, "A systematic and comparative study of distinct recurrent neural networks for lithium-ion battery state-of-charge estimation in electric vehicles," *Energies*, vol. 16, no. 4, p. 2008, Feb. 2023, doi: 10.3390/en16042008.

[36] M. Arif and M. Rashid, "A literature review on model conversion, inference, and learning strategies in EdgeML with TinyML deployment," *Comput., Mater. Continua*, vol. 83, no. 1, pp. 13–64, 2025, doi: 10.32604/cmc.2025.062819.

[37] H. Zhou, X. Zhang, Y. Feng, T. Zhang, and L. Xiong, "Efficient human activity recognition on edge devices using DeepConv LSTM architectures," *Sci. Rep.*, vol. 15, no. 1, p. 13830, 2025, doi: 10.1038/s41598-025-98571-2.

[38] Edge Impulse. (Apr. 29, 2025). *Tips and Tricks: Inference Performance Metrics*. [Online]. Available: https://docs.edgeimpulse.com/docs/tips-and-tricks/inference-performance-metrics

[39] H.-I. Liu, M. Galindo, H. Xie, L.-K. Wong, H.-H. Shuai, Y.-H. Li, and W.-H. Cheng, "Lightweight deep learning for resource-constrained environments: A survey," *ACM Comput. Surv.*, vol. 56, no. 10, pp. 1–42, Oct. 2024, doi: 10.1145/3657282.

[40] P. Li, Z. Ao, J. Hou, S. Xiang, and Z. Wang, "Physics-informed mamba neural network with potential knowledge for state-of-charge estimation of lithium-ion batteries," *J. Power Sources*, vol. 123, Apr. 2025, Art. no. 116546, doi: 10.1016/j.est.2025.116546.

[41] C. She, Y. Li, C. Zou, T. Wik, Z. Wang, and F. Sun, "Offline and online blended machine learning for lithium-ion battery health state estimation," *IEEE Trans. Transport. Electrific.*, vol. 8, no. 2, pp. 1604–1618, Jun. 2022, doi: 10.1109/TTE.2021.3129479.

[42] C. Wang, X. Zhang, X. Yun, and X. Fan, "A novel hybrid machine learning Coulomb counting technique for state of charge estimation of lithium-ion batteries," *J. Energy Storage*, vol. 63, Jul. 2023, Art. no. 107081, doi: 10.1016/j.est.2023.107081.

**SESIDHAR DVSR** received the bachelor's and master's degrees in electronics and communication engineering from Osmania University, Hyderabad, India, in 2002 and 2009, respectively. He is currently pursuing the Ph.D. degree with the Department of Electrical Engineering Science, Ramaiah Institute of Technology, Bengaluru, India, under Visvesvaraya Technological University, Belagavi, India. His research interests include embedded systems, the IoT, and machine learning applied to electric vehicles.

**CHANDRASHEKHAR BADACHI** (Member, IEEE) received the master's degree from Visvesvaraya Technological University, in 2008, and the Ph.D. degree in electrical engineering from Jain University, Bengaluru, India, in 2016. He is currently an Associate Professor with the Department of Electrical Engineering, Ramaiah Institute of Technology, Bengaluru. He holds four patents and his research interests include high-voltage engineering, outdoor installations, mathematical modeling, power systems, electrical vehicular technology, and machine learning applications in electric vehicles.

**CHANDRASHEKAR NAGAWARAM** received the master's degree in digital systems engineering from Osmania University, Hyderabad, India, in 2009. He is currently an Associate Principal Engineer with ABB, Hyderabad. He has eight patents in the field of electronics design. His research interests include embedded systems, analog circuit design, and design for EMI/EMC.

**PANDURANGA CHARY KONDOJU** received the master's degree from JNTU, Hyderabad. He is currently a Software Engineer with RAE, Ford Motor Company, Dearborn, MI, USA. His work focuses on developing next-generation vehicle connectivity software platforms in line with industry standards and crafting scalable and portable in-vehicle and cloud software solutions. He also develops research proposals in vehicle connectivity with universities and external partners. His work has resulted in more than seven patents, many of which have been implemented in the automotive industry.

**C. DHANAMJAYULU** (Senior Member, IEEE) received the B.Tech. degree in electronics and communication engineering from JNTU, Hyderabad, India, the M.Tech. degree in control and instrumentation systems from Indian Institute of Technology Madras, Chennai, India, and the Ph.D. degree in power electronics from Vellore Institute of Technology, Vellore, India. He was invited as a Visiting Researcher with the Department of Energy Technology, Aalborg University, Esbjerg, Denmark, funded by the Danida Mobility Grant, Ministry of Foreign Affairs of Denmark on Denmark's International Development Cooperation. He was a Postdoctoral Fellow with the Department of Energy Technology, Aalborg University, from October 2019 to January 2021. He is currently an Associate Professor with Vellore Institute of Technology. His research interests include machine learning, federated learning, soft computing, computer vision, block chain, multilevel inverters, power converters, active power filters, power quality, grid connected systems, smart grids, and electric vehicle. He is recently recognized as one among the top 2% scientists in the world as per the survey conducted by Elsevier. He is an Academic Editor of the *International Transactions on Electrical Energy Systems* (Wiley), the *Mathematical Problems in Engineering* (Wiley), and the *PLOS One*; an Editorial Member of the *Scientific Reports*; an Associate Editor of the *HardwareX*; an Associate Editor of *Journal of Electrical Engineering and Technology* (Springer); an Editorial Member of the *Discover Applied Sciences*; and an Associate Editor of *e-prime* (Elsevier).

**INNOCENT KAMWA** (Fellow, IEEE) received the Ph.D. degree in electrical engineering from Université Laval in 1989. He is currently a Full Professor with the Department of Electrical Engineering, Tier 1 Canada Research Chair in Decentralised Sustainable Electricity Grids for Smart Communities, and a Distinguished Professor of Power Engineering at Tsinghua University. Previously, he was a Researcher at the Hydro-Québec Research Institute, he was the Principal Scientist of Hydro-Québec's Smart Grid Innovation Program, and the Head of Power Systems and Mathematics, Directing the Network Simulation Laboratory. An international consultant in simulation and stability of power systems, he is a Fellow of the Institute of Electrical and Electronics Engineers (IEEE) For contributions to adaptive power grid control schemes and synchronous generator testing and standards. He is a fellow of the Canadian Academy of Engineering and an International Member of the US National Academy of Engineering. He is also a fellow of the Royal Society of Canada and an editor-in-chief of IEEE *Power and Energy Magazine*. He is the recipient of six IEEE PES Best Paper Prize awards, five IEEE PES Outstanding Working Group awards, the IEEE Charles-Proteus-Steinmetz prize, and the IEEE Power and Energy Society's Charles Concordia Power Systems Engineering prize.